

Dynamic Dispatch and Type Inference Semipredicates

David Chase

Oracle Labs

david.r.chase@oracle.com

Justin Hilburn

Oracle Labs

justin.hilburn@oracle.com

Victor Luchangco

Oracle Labs

victor.luchangco@oracle.com

Karl Naden

Oracle Labs

karl.naden@oracle.com

Sukyoung Ryu

KAIST

sryu.cs@kaist.ac.kr

Guy L. Steele Jr.

Oracle Labs

guy.steele@oracle.com

Jean-Baptiste Tristan

Oracle Labs

jean.baptiste.tristan@oracle.com

Abstract

In an object-oriented programming language with fully symmetric dynamic multimethod dispatch, it is frequently necessary to decide at run time whether a given method is applicable to a given set of arguments. If furthermore the language features parametrically polymorphic types whose parameters are not “erased” at run time, it may be necessary, as part of the run-time determination of applicability, to compute, based on the ilks (“run-time types”) of the argument values, a type for each type parameter. Thus the applicability test is a semipredicate: if it does not return *false*, then it returns not just *true*, but some useful information—in this case, a substitution of types for type parameters that validates the judgment of applicability. Because this process may be required for every method call, it is desirable for this process to be efficient.

We present algorithms for dispatch semipredicates in Fortress, a statically typed object-oriented programming language with type-parametric traits, objects, functions, and methods, symmetric multiple inheritance, and fully symmetric multiway dispatch for overloaded functions as well as overloaded methods. Type parameters of traits can be covariant, contravariant, or invariant. We explore some design tradeoffs; for example, certain difficult cases that are costly to process can be avoided if either contravariance or union types are eliminated from the model. To provide a rigorous expository framework for this presentation, we formally define Welterweight Fortress, a model much larger than the previously reported Featherweight Fortress.

Categories and Subject Descriptors D.3.3 [Programming Languages]: Language Constructs and Features—classes and objects, inheritance, modules, packages, polymorphism

General Terms Languages

Keywords object-oriented programming, multiple dispatch, symmetric dispatch, multiple inheritance, overloading, ilks, run-time types, static types, components, modularity, meet rule, methods, multimethods, separate compilation, Fortress

1. Contributions

The contributions of this paper are:

(1) To define Welterweight Fortress, which goes beyond Featherweight Fortress [8] to present a substantial model of a statically typed object-oriented programming language with multiple inheritance of type-parametric methods from type-parametric traits whose type parameters can be covariant, contravariant, or invariant; type-parametric functions; abstract functions and methods; explicitly declared “excludes” and “comprises” type relationships; and fully symmetric multimethod dispatch. This model includes a complete formal codification of the No Duplicates Rule, the Meet Rule, and the Return Type Rule discussed in [3], which, as part of static type checking, guarantee that at run time a dynamically most specific applicable visible method or function always exists and is unique.

(2) To present an improved, more practical formulation of the Return Type Rule.

(3) To introduce an “Ancestor Meet Rule” that, together with left-to-right scoping of the type parameters of methods and functions, enables an implementation of fully symmetric multimethod dispatch that is almost always quite efficient.

(4) The real meat of the paper, for which Welterweight Fortress serves as the expository framework: To describe in detail algorithms for determining, at run time, whether a type-parametric function or method is applicable to a given tuple of argument values, and if so, what types should be assigned to the type parameters during execution of the function or method. This matters because in Fortress, unlike Java [7] or Scala [13], generic types are not erased. We characterize cases in which exponential search may be required and argue that they are unlikely to arise in practice.

(5) In passing, to describe a more rigorous and expressive version of the customary “underline” metanotation. Use of underlines and special overarrows allows a more compact and more accurate presentation of complex judgments.

$\Pi ::= \overline{\psi}, \overline{\varphi}, e$	program (declarations plus expression)	$\tau ::= P$	type parameter reference
$\psi ::= \text{trait } T[\overline{V\beta}] <: \{\overline{t}\} \equiv \bigcup \{\overline{c}\} \diamond \{\overline{t}\} \overline{\mu} \text{ end}$	trait declaration	$ c$	constructed type
$ \text{trait } T[\overline{V\beta}] <: \{\overline{t}\} \diamond \{\overline{t}\} \overline{\mu} \text{ end}$	trait declaration	$ (\overline{\tau})$	tuple type
$ \text{object } O[\overline{\beta}] (\overline{x}:\overline{\tau}) <: \{\overline{t}\} \overline{\mu} \text{ end}$	object declaration	$ (\tau \rightarrow \tau)$	arrow type
$V ::= \text{covariant} \text{contravariant} \text{invariant}$	variance	$ \text{Any}$	special Any type
$\varphi ::= f[\overline{\beta}] (\overline{x}:\overline{\tau}) : \tau = e$	function definition	$ \text{Object}$	special Object type
$ f[\overline{\beta}] (\overline{x}:\overline{\tau}) : \tau$	abstract function declaration	$c ::= O[\overline{\tau}]$	object type
$\mu ::= m[\overline{\kappa}] (\overline{x}:\overline{\tau}) : \tau = e$	dotted method definition	$ t$	trait type
$ m[\overline{\kappa}] (\overline{x}:\overline{\tau}) : \tau$	abstract dotted method declaration	$t ::= T[\overline{\tau}]$	trait type
$\beta ::= P <: \{\overline{\tau}\}$	simple type parameter binding	$P ::= \text{identifier}$	type parameter name
$\kappa ::= \{\overline{\tau}\} <: P <: \{\overline{\tau}\}$	complete type parameter binding	$T ::= \text{identifier}$	generic trait name
$e ::= x$	variable or field reference	$O ::= \text{identifier}$	generic object name
$ \text{self}$	self reference	$x ::= \text{identifier}$	variable or field name
$ (\overline{e})$	tuple creation	$f ::= \text{identifier}$	function name
$ \pi_i(e)$	tuple projection	$m ::= \text{identifier}$	method name
$ ((\overline{x}:\overline{\tau}) : \tau \Rightarrow e)$	closure creation	$Id ::= P$	names
$ e @ (\overline{e})$	closure application	$ T$	
$ O[\overline{\tau}] (\overline{e})$	object creation	$ O$	
$ f(\overline{e})$	function invocation	$ x$	
$ e.m(\overline{e})$	method dotted invocation	$ f$	
$ (e \text{ match } x: \tau \Rightarrow e \text{ else } e)$	match expression	$ m$	

Figure 1. Grammar for Welterweight Fortress

2. Notation

An overbar indicates that the expression or token string beneath it may appear any number of times, including zero; if there are two or more copies, they are separated by commas. Nonterminals from the BNF grammars may be expanded differently in each copy, except that an underlined portion is expanded the *same* way in each copy. If two or more overbar constructions appear within a single context (such as an inference rule or expository paragraph) and the same BNF nonterminal appears in both, then each of the overbar constructions must produce the same number of copies, and corresponding copies will expand the BNF nonterminal in the same way.

A novel “overarrow” notation is used in Figure 6 in two idiomatic ways: (1) $\Delta = \{\overline{B}\}$ defines a series of Δ values, each a set whose elements are a prefix of the sequence $\overline{B}$; if the overbar construction produces four copies, then there are four instances of Δ , and the overall idiom produces the four items $\Delta_1 = \{\}$, $\Delta_2 = \{B_1\}$, $\Delta_3 = \{B_1, B_2\}$, $\Delta_4 = \{B_1, B_2, B_3\}$; and (2) $\overline{d \text{ meets } d}$ expands to a copy of d meets d , for every pair of distinct d instances—if the overbar construction produces four copies, for example, then there are four instances of d , and the overall idiom produces the six items d_1 meets d_2 , d_1 meets d_3 , d_2 meets d_3 , d_1 meets d_4 , d_2 meets d_4 , d_3 meets d_4 . Appendix A gives a precise description of our notation.

3. Grammar

The grammar for Welterweight Fortress is given in Figure 1. Metavariables defined by this grammar and also in later figures are listed in Figure 2.

A program consists of declarations and an expression to be evaluated in the context of those declarations. Each declaration defines a trait, an object, or a top-level function.

A trait is similar to a Java interface, but can contain method declarations. It also has an *extends clause* $<: \{\overline{t}\}$, an optional *comprises clause* $\equiv \bigcup \{\overline{c}\}$, and an *excludes clause* $\diamond \{\overline{t}\}$. The extends clause indicates what other trait instances are supertypes of the trait; the comprises clause, if present, indicates that no value can belong to the trait unless it also belongs to one of the comprised types; and the excludes clause indicates what other trait instances cannot have values in common with this trait.

An object is similar to a Java class. Rather than having a separately declared constructor method, it has a parameter list containing names of fields and their types; an object creation expression provides a set of arguments that are simply used to initialize the declared fields, with no further action taken. An object declaration also has an extends clause, indicating what trait instances are supertypes of the object type.

A top-level function definition has a parameter list containing names of parameters and their types, and also a return type and a body expression. A top-level abstract function declaration is similar, but has no body expression. Top-

Metavariables for BNF nonterminals defined in Figure 1:

Π ranges over programs Π
 ψ ranges over top-level declarations ψ
 V ranges over variances V
 φ ranges over function declarations φ
 μ ranges over dotted method declarations μ
 β ranges over simple type parameter bindings β
 κ ranges over complete type parameter bindings κ
 e ranges over expressions e
 t ranges over trait types t
 c ranges over constructed types c
 P, Q, S range over type parameter names P
 T, U, G, H, J range over trait names T
 O ranges over object names O
 x ranges over variable/field names x
 f ranges over function names f
 m ranges over method names m
 τ, ζ, ξ, ω range over expressible types τ

Metavariables for BNF nonterminals defined in Figure 3:

$\alpha, \gamma, \rho, \chi, \eta$ range over lattice types α
 C ranges over lattice constructed types C
 Ξ ranges over existentially quantified types Ξ
 Υ ranges over universally quantified types Υ
 B ranges over lattice type parameter bindings B
 δ ranges over type-declaration environment entries δ
 I ranges over inference variable names I
 b ranges over bindable value names b
 σ ranges over type/parameter substitutions σ
 n ranges over generalized procedure names n
 d ranges over procedure declarations d
 D ranges over fully instantiated procedure declarations D
 Δ ranges over type-declaration environments Δ
 Φ ranges over function-declaration environments Φ
 Γ ranges over variable-type environments Γ
 K ranges over instantiated complete bindings K
 F ranges over instantiated functions F
 N ranges over trait-instantiated methods N
 M ranges over fully instantiated methods M
 u ranges over trait-instantiated expressions u
 ϵ ranges over fully instantiated expressions ϵ

Metavariables for BNF nonterminals defined in Figure 9:

E ranges over evaluation contexts E
 R ranges over redexes R
 g ranges over ground types g
 k ranges over ilks k
 v ranges over values v
 X ranges over ground constructed types X
 s ranges over signature types s
 p ranges over parametric signature types p

Metavariables for BNF nonterminals defined in Figure 12:

C ranges over type constraints C

π does not range over any BNF nonterminal

Figure 2. Metavariables Used in This Paper

$\alpha ::= P$	type parameter name
$ C$	lattice constructed type
$ (\bar{\alpha})$	tuple type
$ (\alpha \rightarrow \alpha)$	arrow type
$ \text{Any}$	special Any type
$ \text{Object}$	special Object type
$ \text{Bottom}$	special Bottom type
$ (\alpha \cup \alpha)$	union type
$ (\alpha \cap \alpha)$	intersection type
$C ::= T[\bar{\alpha}]$	lattice trait type
$ O[\bar{\alpha}]$	lattice object type
$\Xi ::= \exists[K]\alpha$	existentially quantified type
$\Upsilon ::= \forall[K]\alpha$	universally quantified type
$B ::= \{\bar{\alpha}\} <: V P <: \{\bar{\alpha}\}$	environment parameter binding
$\delta ::= \psi$	program declaration
$ B$	environment parameter binding
$I ::= \text{identifier}$	type inference variable name
$b ::= x$	variable name
$ \text{self}$	self keyword
$\sigma ::= [\bar{\alpha}/\bar{P}]$	type/parameter substitution
$n ::= f$	function name
$ (C, m)$	lattice type and method name
$d ::= \varphi$	function declaration
$ (C, M)$	method instance
$D ::= F$	instantiated function
$ (X, N)$	fully instantiated method
$\Delta ::= \{\bar{\delta}\}$	type-declaration environment
$\Phi ::= \{\bar{\varphi}\}$	function-declaration environment
$\Gamma ::= \bar{b}: \bar{\alpha}$	variable-type environment
$K ::= \{\bar{\alpha}\} <: P <: \{\bar{\alpha}\}$	instantiated complete binding
$F ::= f[\bar{\alpha}](\bar{x}: \bar{g}): g = \epsilon$	instantiated function
$M ::= m[\bar{K}](\bar{x}: \bar{\alpha}): \alpha = u$	trait-instantiated method
$N ::= m[\bar{\alpha}](\bar{x}: \bar{g}): g = \epsilon$	fully instantiated method
$u ::= x$	trait-instantiated expression
$ \text{self}$	
$ (\bar{u})$	
$ \pi_i(u)$	
$ ((\bar{x}: \bar{\alpha}): \alpha \Rightarrow u)$	
$ u \otimes (\bar{u})$	
$ O[\bar{\alpha}](\bar{u})$	
$ f(\bar{u})$	
$ u.m(\bar{u})$	
$ (u \text{ match } x: \alpha \Rightarrow u \text{ else } u)$	
$\epsilon ::= x$	fully instantiated expression
$ \text{self}$	
$ (\bar{\epsilon})$	
$ \pi_i(\epsilon)$	
$ ((\bar{x}: \bar{g}): g \Rightarrow \epsilon)$	
$ \epsilon \otimes (\bar{\epsilon})$	
$ O[\bar{g}](\bar{\epsilon})$	
$ f(\bar{\epsilon})$	
$ \epsilon.m(\bar{\epsilon})$	
$ (\epsilon \text{ match } x: g \Rightarrow \epsilon \text{ else } \epsilon)$	

Figure 3. Symbols Not Used in the Concrete Syntax

Well-formed types: $\Delta \vdash \alpha \text{ ok} \quad \Delta \vdash \Xi \text{ ok} \quad \Delta \vdash \Upsilon \text{ ok}$

$$\begin{array}{c}
\frac{\{\bar{_}\} <: \text{invariant } P <: \{\bar{_}\} \in \Delta}{\Delta \vdash P \text{ ok}} \quad [\text{W-PARAM-IN}] \\
\\
\frac{\{\bar{_}\} <: \text{covariant } P <: \{\bar{_}\} \in \Delta}{\Delta \vdash P \text{ ok}} \quad [\text{W-PARAM-CO}] \\
\\
\frac{\text{trait } T \llbracket V P <: \{\bar{\xi}\} \rrbracket \text{ -- end} \in \Delta \quad \#(\bar{\alpha}) = \#(\bar{P}) \quad \text{flip}(V, \Delta) \vdash \alpha \text{ ok}}{\Delta \vdash \alpha <: \llbracket \alpha / \bar{P} \rrbracket \xi} \quad [\text{W-TRAIT}] \\
\\
\frac{\text{object } O \llbracket P <: \{\bar{\xi}\} \rrbracket \text{ -- end} \in \Delta \quad \#(\bar{\alpha}) = \#(\bar{P}) \quad \Delta \vdash \bar{\alpha} \text{ ok}}{\Delta \vdash \alpha <: \llbracket \alpha / \bar{P} \rrbracket \xi} \quad [\text{W-OBJECT}] \\
\\
\frac{\Delta \vdash \bar{\alpha} \text{ ok}}{\Delta \vdash (\bar{\alpha}) \text{ ok}} \quad [\text{W-TUPLE}] \\
\\
\frac{\text{flip}(\Delta) \vdash \alpha \text{ ok} \quad \Delta \vdash \rho \text{ ok}}{\Delta \vdash (\alpha \rightarrow \rho) \text{ ok}} \quad [\text{W-ARROW}] \\
\\
\Delta \vdash \text{Any ok} \quad [\text{W-ANY-TYPE}] \\
\Delta \vdash \text{Object ok} \quad [\text{W-OBJECT-TYPE}] \\
\Delta \vdash \text{Bottom ok} \quad [\text{W-BOTTOM-TYPE}] \\
\\
\frac{\Delta \vdash \alpha \text{ ok} \quad \Delta \vdash \gamma \text{ ok}}{\Delta \vdash (\alpha \cup \gamma) \text{ ok} \quad \Delta \vdash (\alpha \cap \gamma) \text{ ok}} \quad [\text{W-UNION-INT}] \\
\\
\frac{\Delta \vdash \bar{\chi} \text{ ok} \quad \Delta \vdash \bar{\eta} \text{ ok}}{\Delta \cup \{\{\bar{\chi}\} <: \text{invariant } P <: \{\bar{\eta}\}\} \vdash \alpha \text{ ok}} \quad [\text{W-EXISTS}] \\
\frac{\Delta \vdash \exists \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket \alpha \text{ ok}}{\Delta \vdash \exists \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket \alpha \text{ ok}} \quad [\text{W-EXISTS}] \\
\\
\frac{\Delta \vdash \bar{\chi} \text{ ok} \quad \Delta \vdash \bar{\eta} \text{ ok}}{\Delta \cup \{\{\bar{\chi}\} <: \text{invariant } P <: \{\bar{\eta}\}\} \vdash \alpha \text{ ok}} \quad [\text{W-FORALL}] \\
\frac{\Delta \vdash \forall \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket \alpha \text{ ok}}{\Delta \vdash \forall \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket \alpha \text{ ok}} \quad [\text{W-FORALL}]
\end{array}$$

Variance flip: $\text{flip}(\Delta) = \Delta \quad \text{flip}(V) = V \quad \text{flip}(V, \Delta) = \Delta$

$$\begin{aligned}
\text{flip}(\Delta) &= \bigcup_{\delta \in \Delta} \text{oneFlip}(\delta) \\
\text{oneFlip}(\{\bar{\chi}\} <: V P <: \{\bar{\eta}\}) &= \{\{\bar{\chi}\} <: \text{flip}(V) P <: \{\bar{\eta}\}\} \\
\text{oneFlip}(\psi) &= \{\psi\} \\
\text{flip}(\text{covariant}) &= \text{contravariant} \\
\text{flip}(\text{contravariant}) &= \text{covariant} \\
\text{flip}(\text{invariant}) &= \text{invariant} \\
\text{flip}(\text{covariant}, \Delta) &= \Delta \\
\text{flip}(\text{contravariant}, \Delta) &= \text{flip}(\Delta) \\
\text{flip}(\text{invariant}, \Delta) &= \text{flat}(\Delta) \quad (\text{see Figure 6})
\end{aligned}$$

Figure 4. Well-formed Types

Well-formed expressions, static types: $\Delta; \Phi; \Gamma \vdash e : \alpha$

$$\begin{array}{c}
\frac{\Gamma = \bar{_}, b : \alpha, \bar{b}' : \bar{_} \quad b \notin \{\bar{b}'\}}{\Delta; \Phi; \Gamma \vdash b : \alpha} \quad [\text{T-VARIABLE}] \\
\\
\frac{\Delta; \Phi; \Gamma \vdash \bar{e} : \bar{\alpha}}{\Delta; \Phi; \Gamma \vdash (\bar{e}) : (\bar{\alpha})} \quad [\text{T-TUPLE}] \\
\\
\frac{\Delta; \Phi; \Gamma \vdash e : (\bar{\alpha})}{\Delta; \Phi; \Gamma \vdash \pi(\bar{e}) : \alpha} \quad [\text{T-PROJECT}] \\
\\
\frac{\text{distinct}(\bar{x}) \quad \Delta \vdash \bar{\tau} \text{ ok} \quad \Delta \vdash \omega \text{ ok} \quad \Delta; \Phi; \Gamma, \bar{x} : \bar{\tau} \vdash e : \rho \quad \Delta \vdash \rho <: \omega}{\Delta; \Phi; \Gamma \vdash ((\bar{x} : \bar{\tau}) : \omega \Rightarrow e) : ((\bar{\tau}) \rightarrow \omega)} \quad [\text{T-CLOSURE}] \\
\\
\frac{\Delta; \Phi; \Gamma \vdash e : ((\bar{\alpha}) \rightarrow \rho) \quad \Delta; \Phi; \Gamma \vdash (\bar{e}') : (\bar{\chi}) \quad \Delta \vdash \bar{\chi} <: \alpha}{\Delta; \Phi; \Gamma \vdash e \mathcal{Q}(\bar{e}') : \rho} \quad [\text{T-APPLY}] \\
\\
\frac{\text{object } O \llbracket P <: \{\bar{_}\} \rrbracket (\bar{x} : \bar{\omega}) \text{ -- end} \in \Delta \quad \Delta \vdash O[\bar{\tau}] \text{ ok} \quad \Delta; \Phi; \Gamma \vdash \bar{e} : \bar{\eta} \quad \Delta \vdash \bar{\eta} <: \llbracket \bar{\tau} / \bar{P} \rrbracket \omega}{\Delta; \Phi; \Gamma \vdash O[\bar{\tau}](\bar{e}) : O[\bar{\tau}]} \quad [\text{T-OBJECT}] \\
\\
\frac{\text{msav}(\Delta, \Phi, f, (\bar{\eta})) = \{f \llbracket \bar{_} \rrbracket (\bar{_}) : \omega \bar{_}\} \quad \Delta; \Phi; \Gamma \vdash \bar{e} : \bar{\eta}}{\Delta; \Phi; \Gamma \vdash f(\bar{e}) : \omega} \quad [\text{T-FUNCTION}] \\
\\
\frac{\text{msav}(\Delta, \Phi, (C, m), (\bar{\eta})) = \{m \llbracket \bar{_} \rrbracket (\bar{_}) : \omega \bar{_}\} \quad \Delta; \Phi; \Gamma \vdash e : C \quad \Delta; \Phi; \Gamma \vdash \bar{e}' : \bar{\eta}}{\Delta; \Phi; \Gamma \vdash e.m(\bar{e}') : \omega} \quad [\text{T-METHOD}] \\
\\
\frac{\Delta; \Phi; \Gamma \vdash e : \alpha \quad \Delta \vdash \tau \text{ ok} \quad \Delta; \Phi; \Gamma, x : (\alpha \cap \tau) \vdash e' : \eta \quad \Delta; \Phi; \Gamma \vdash e'' : \chi}{\Delta; \Phi; \Gamma \vdash (e \text{ match } x : \tau \Rightarrow e' \text{ else } e'') : (\eta \cup \chi)} \quad [\text{T-MATCH}]
\end{array}$$

Figure 5. Static Types of Expressions

level function declarations may be overloaded; that is, more than one function declaration may have the same name f .

Method definitions appearing in trait or object declarations are similar in form to function definitions; however, the keyword `self` may be used within its body expression to refer to a value (the *target object*) for which the method was invoked. An abstract method declaration is similar, but has no body expression. Method declarations may be overloaded; that is, more than one method declaration may have the same name m .

Traits, objects, functions, and methods all have a (possibly empty) list of *static type parameters*. Associated with each static type parameter is (possibly empty) set of *upper bounds*; any type used to instantiate the type parameter must be a subtype of each of the declared upper bounds. Each

Well-formed program and its static type: $\boxed{\vdash \Pi : g}$

$$\begin{array}{c}
\Pi = \overline{\psi}, \overline{\varphi}, e \quad \text{distinct}(\overline{\psi}) \quad \text{distinct}(\overline{\varphi}) \quad \Delta = \{\overline{\psi}\} \quad \Phi = \{\overline{\varphi}\} \quad \#(\text{typeNameNames}(\Delta)) = \#(\overline{\psi}) \quad \Delta, \Phi \vdash \overline{\psi} \text{ ok} \\
\Delta, \Phi \vdash \overline{\varphi} \text{ ok} \quad \{\overline{\varphi}'\} = \text{abstractDecls}(\{\overline{\varphi}\}) \quad \Delta \vdash \overline{\varphi}' \text{ covered by } \text{concreteDecls}(\{\overline{\varphi}\}) \quad \Delta; \Phi; \bullet \vdash e : g \\
\Delta \vdash \overline{\varphi} \text{ not duplicate of } \underline{\varphi} \quad \Delta \vdash \overline{\varphi} \text{ meet } \underline{\varphi} \text{ ok} \quad \Delta \vdash \overline{\varphi} \text{ return type wrt } \underline{\varphi} \text{ ok} \quad \Delta \vdash \underline{\varphi} \text{ return type wrt } \overline{\varphi} \text{ ok} \\
\hline
\vdash \Pi : g
\end{array}$$

[T-PROGRAM]

Well-formed trait and object declarations: $\boxed{\Delta, \Phi \vdash \psi \text{ ok}}$

$$\begin{array}{c}
\Delta' = \Delta \cup \{\{\} <: V P <: \{\overline{\xi}\}\} \quad \text{distinct}(\overline{P}) \quad \Delta' \vdash \overline{\xi} \text{ ok} \quad \{\overline{\chi}\} = \text{properAncestors}(\Delta', T[\overline{P}]) \\
\Delta' \vdash \overline{t} \text{ ok} \quad \text{name}(t') \neq \overline{t} \quad \Delta' \vdash \overline{t'} \text{ ok} \quad T \notin \text{typeNameNames}(\{\overline{\chi}\}) \quad \text{distinct}(\overline{\mu}) \quad \Delta; \overline{\chi} \vdash \overline{\chi} \text{ and } \underline{\chi} \text{ ancestors ok} \\
\boxed{\text{name}(c) \neq \overline{t} \quad \Delta' \vdash \overline{c} \text{ ok} \quad \forall t'' \in \{\overline{t'}\}. (\Delta' \vdash \overline{c} \not<: t'') \quad \forall t'' \in \{\overline{t}\}. (\Delta' \vdash \overline{t'} \not<: c) \quad \Delta' \vdash \overline{c} <: T[\overline{P}]}} \\
\Delta' \vdash T[\overline{P}] \not<: t' \quad \Delta'; \Phi; \text{self}: T[\overline{P}]; \overline{P} \vdash \overline{\mu} \text{ ok} \quad \{\overline{d}\} = \{(C, M)\} = \text{allVisible}(\Delta', T[\overline{P}]) \\
\Delta \vdash \overline{d} \text{ not duplicate of } \underline{d} \quad \Delta \vdash \overline{d} \text{ meet } \underline{d} \text{ ok} \quad \Delta \vdash \overline{d} \text{ return type wrt } \underline{d} \text{ ok} \quad \Delta \vdash \underline{d} \text{ return type wrt } \overline{d} \text{ ok} \\
\hline
\Delta, \Phi \vdash \text{trait } T[\overline{V P <: \{\overline{\xi}\}}] <: \{\overline{t}\} \equiv \bigcup \{\overline{c}\} \diamond \{\overline{t'}\} \overline{\mu} \text{ end ok}
\end{array}$$

[D-TRAIT]

$$\begin{array}{c}
\Delta' = \Delta \cup \{\{\} <: \text{invariant } P <: \{\overline{\xi}\}\} \quad \text{distinct}(\overline{P}) \quad \Delta' \vdash \overline{\xi} \text{ ok} \quad \{\overline{\chi}\} = \text{properAncestors}(\Delta', O[\overline{P}]) \\
\Delta' \vdash \overline{\tau} \text{ ok} \quad \Delta' \vdash \overline{t} \text{ ok} \quad \text{distinct}(\overline{x}) \quad \text{distinct}(\overline{\mu}) \quad \Delta; \overline{\chi} \vdash \overline{\chi} \text{ and } \underline{\chi} \text{ ancestors ok} \\
\Delta'; \text{self}: O[\overline{P}], \overline{x}: \overline{\tau}; \overline{P} \vdash \overline{\mu} \text{ ok} \quad \{\overline{d}\} = \{(C, M)\} = \text{allVisible}(\Delta', O[\overline{P}]) \\
\{(C', M')\} = \text{abstractDecls}(\{\overline{d}\}) \quad \Delta \vdash (C', M') \text{ covered by } \text{concreteDecls}(\{\overline{d}\}) \\
\Delta \vdash \overline{d} \text{ not duplicate of } \underline{d} \quad \Delta \vdash \overline{d} \text{ meet } \underline{d} \text{ ok} \quad \Delta \vdash \overline{d} \text{ return type wrt } \underline{d} \text{ ok} \quad \Delta \vdash \underline{d} \text{ return type wrt } \overline{d} \text{ ok} \\
\hline
\Delta, \Phi \vdash \text{object } O[\overline{P <: \{\overline{\xi}\}}] (\overline{x}: \overline{\tau}) <: \{\overline{t}\} \overline{\mu} \text{ end ok}
\end{array}$$

[D-OBJECT]

Well-formed function declarations: $\boxed{\Delta, \Phi \vdash \varphi \text{ ok}}$

$$\begin{array}{c}
\Delta' = \Delta \cup \{\{\} <: \text{invariant } P <: \{\overline{\xi}\}\} \quad \Delta'' = \Delta \cup \{\{\} <: \text{invariant } P <: \{\overline{\xi}\}\} \quad \text{distinct}(\overline{P}) \quad \text{distinct}(\overline{x}) \\
\text{flip}(\Delta') \vdash \overline{\xi} \text{ ok} \quad \text{flip}(\Delta'') \vdash \overline{\tau} \text{ ok} \quad \Delta'' \vdash \omega \text{ ok} \quad \boxed{\text{flat}(\Delta''); \Phi; \overline{x}: \overline{\tau} \vdash e : \rho \quad \Delta'' \vdash \rho <: \omega} \\
\hline
\Delta, \Phi \vdash f[\overline{P <: \{\overline{\xi}\}}] (\overline{x}: \overline{\tau}): \omega \boxed{= e} \text{ ok}
\end{array}$$

[D-FUNCTION]

Well-formed method declarations: $\boxed{\Delta; \Phi; \Gamma; \overline{P} \vdash \mu \text{ ok}}$

$$\begin{array}{c}
\Delta' = \Delta \cup \{\{\overline{\zeta}\} <: \text{invariant } Q <: \{\overline{\xi}\}\} \quad \Delta'' = \Delta \cup \{\{\overline{\zeta}\} <: \text{invariant } Q <: \{\overline{\xi}\}\} \quad \text{distinct}(\overline{P}, \overline{Q}) \quad \text{distinct}(\overline{x}) \\
\Delta \vdash \overline{\zeta} \text{ ok} \quad \text{flip}(\Delta') \vdash \overline{\xi} \text{ ok} \quad \text{flip}(\Delta'') \vdash \overline{\tau} \text{ ok} \quad \Delta'' \vdash \omega \text{ ok} \quad \boxed{\text{flat}(\Delta''); \Phi; \Gamma; \overline{x}: \overline{\tau} \vdash e : \rho \quad \Delta'' \vdash \rho <: \omega} \\
\hline
\Delta; \Phi; \Gamma; \overline{P} \vdash m[\overline{\{\overline{\zeta}\} <: Q <: \{\overline{\xi}\}}] (\overline{x}: \overline{\tau}): \omega \boxed{= e} \text{ ok}
\end{array}$$

[D-METHOD]

Variance flattening: $\boxed{\text{flat}(\Delta) = \Delta}$

$$\begin{array}{c}
\text{flat}(\Delta) = \bigcup_{\delta \in \Delta} \text{oneFlat}(\delta) \quad \text{oneFlat}(\{\overline{\chi}\} <: V P <: \{\overline{\eta}\}) = \{\{\overline{\chi}\} <: \text{invariant } P <: \{\overline{\eta}\}\} \\
\text{oneFlat}(\psi) = \{\psi\}
\end{array}$$

Figure 6. Program typing and Well-formed Definitions

Covering an abstract declaration: $\Delta \vdash d$ covered by $\{\bar{d}\}$

$$\frac{\text{noSelfDom}(d) = \exists \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket \tau \quad \text{distinct}(\bar{P})}{\Delta \vdash \text{noSelfDom}(d') \sqsubseteq \exists \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket (\bigcup \{\bar{\tau}\})} \quad \Delta \vdash d' \text{ covered by } \{\bar{d}\} \quad [\text{COVERED}]$$

Selfless domain of a declaration: $\text{noSelfDom}(d) = \Xi$

$$\begin{aligned} \text{noSelfDom}(f \llbracket P <: \{\bar{\xi}\} \rrbracket (\bar{x}:\bar{\tau}) : \omega = e) \\ &= \exists \llbracket \{\bar{\xi}\} <: P <: \{\bar{\xi}\} \rrbracket (\bar{\tau}) \\ \text{noSelfDom}((C, m \llbracket \{\bar{\xi}\} <: P <: \{\bar{\xi}\} \rrbracket (\bar{x}:\bar{\tau}) : \omega = e)) \\ &= \exists \llbracket \{\bar{\xi}\} <: P <: \{\bar{\xi}\} \rrbracket (\bar{\tau}) \end{aligned}$$

Concrete and abstract declarations

$$\begin{aligned} \text{concreteDecls}(\{\bar{d}\}) &= \{d' \mid d' \in \{\bar{d}\}, \text{concrete}(d')\} \\ \text{abstractDecls}(\{\bar{d}\}) &= \{d' \mid d' \in \{\bar{d}\}, \neg \text{concrete}(d')\} \\ \text{concrete}(d) &= (d = f _ = e) \vee (d = (_, m _ = e)) \end{aligned}$$

Figure 7. Determining whether an abstract declaration is “covered” by concrete implementations. Alpha-renaming of type variables may be necessary to apply these rules.

static type parameter of a method also has a (possibly empty) set of *lower bounds*; any type used to instantiate the type parameter must be a supertype of each of the declared lower bounds. Each static type parameter of a trait has an associated *variance*, which indicates ways in which different instances of the same trait may extend or exclude each other.

Most of the forms of expression are fairly conventional. The function creation expression might be called a “typed lambda expression” in other languages. We use the symbol @ to distinguish application of function-typed values from invocation of (possibly overloaded) top-level functions. Function invocations and method invocations each come in two forms, depending on whether static type arguments are provided explicitly or are to be inferred. The *match expression* is a kind of compromise between a conventional if expression and a conventional typecase expression. One can get the effect of if e_1 then e_2 else e_3 by declaring objects $\text{True}[]$ and $\text{False}[]$, using them as values of predicates, and then writing $e_1 \text{ match } x': \text{True}[] \Rightarrow e_2 \text{ else } e_3$.

The type Any is a supertype of all types. The type Object is a supertype of every constructed type, that is, every trait type and every object type. (The BNF nonterminal *Id* is not used in the grammar, but is used in later figures.)

4. Wellformedness and Static Typing

Figure 3 gives a grammar for certain symbols that do not appear in the concrete grammar of Figure 1 but are used in later figures. (Yet other symbols are defined in Figure 9.)

Figure 4 gives rules for deciding whether a type is well-formed, given a set of type declarations Δ . These rules also perform *variance checking*, similar to that performed by Scala [13], to ensure that a covariant or contravariant type parameter is used only in appropriate contexts. That is why the rules allow a type parameter to be used only if it is marked *invariant* or *covariant*. In other places, the rules use the function *flip* to invert the variance of covariant and contravariant parameters in the type environment before checking certain parts of the type expression, such as the domain of an arrow type, which is a contravariant context.

Figure 5 gives rules for deciding whether an expression is well-formed, given a set of type declarations Δ , a set of function declarations Φ , and a variable-type environment Γ . These rules also compute the static type of the expression. Note that rule [T-PROJECT] effectively defines tuple projection functions $\pi_1, \pi_2, \pi_3, \dots$ for tuples of all possible lengths. The rule [T-MATCH] is of interest because it explicitly introduces both intersection and union types into the type calculus (though intersections and unions can also be introduced implicitly in other parts of the language). The most interesting rules in this figure are [D-FUNCTION] and [D-METHOD], which use a function *msav* (defined later in Figure 10) to find the most specific applicable visible function or method for that call site. Note that the last argument to *msav* in these rules is a tuple of the static types of the argument expressions; therefore these uses of *msav* identify the *statically* most specific applicable visible function or method.

Figure 6 gives rules for deciding whether declarations are well-formed and well-typed, given the overall set of declarations Δ , and for determining the type of a program.

The rule [T-PROGRAM] verifies that no two declarations are identical, that declared traits and objects all have distinct names, and that the declarations and the expression are well-formed. It also verifies that any abstract function declarations are “covered” by concrete implementations. The notion of “covering” is a generalization of the notion of “overriding,” because an abstract function with an argument that is a trait type with a *comprises* clause is considered to be covered if it is implemented for all the types comprised. For example:

```
trait Color[] <: {} ≡ ∪ {Red[], Green[]} ◇ {} end
trait Green[]() <: {Color[]} end
trait Red[]() <: {Color[]} end
toggle[](x : Color[]) : Color[]
toggle[](y : Green[]) : Color[] = Red[]()
toggle[](y : Red[]) : Color[] = Green[]()
```

The abstract function toggle that takes an argument of type Color[] is considered to be implemented by the two concrete definitions of toggle because they collectively accept all the types comprised by Color[]. The rules for checking such covering are actually more general than that, and appear in Figure 7. The basic idea is that the domain type of an

Distinct names: $\boxed{\text{distinct}(\overline{Id})}$

$$\frac{|\{\overline{Id}\}| = \#(\overline{Id})}{\text{distinct}(\overline{Id})}$$

Distinct declarations: $\boxed{\text{distinct}(\overline{\psi}) \quad \text{distinct}(\overline{\varphi})}$

$$\frac{|\{\overline{\psi}\}| = \#(\overline{\psi})}{\text{distinct}(\overline{\psi})} \quad \frac{|\{\overline{\varphi}\}| = \#(\overline{\varphi})}{\text{distinct}(\overline{\varphi})}$$

Name of function or method: $\boxed{\text{name}(d) = Id}$

$$\text{name}(f[\overline{\square}](-): -) = f \quad \text{name}((-, m[\overline{\square}](-): -)) = m$$

Name of constructed type: $\boxed{\text{name}(C) = Id}$

$$\text{name}(T[\overline{\square}]) = T \quad \text{name}(O[\overline{\square}]) = O$$

Name of a trait, object, or parameter: $\boxed{\text{name}(\delta) = Id}$

$$\begin{aligned} \text{name}(\text{trait } T[\overline{\square}] \text{ end}) &= T \\ \text{name}(\text{object } O[\overline{\square}] \text{ end}) &= O \\ \text{name}(\{\overline{\square} <: V \ P <: \{\overline{\square}\}\}) &= P \end{aligned}$$

Methods of constructed type: $\boxed{\text{methods}(\Delta, C) = \{(\overline{C}, \overline{M})\}}$

$$\begin{aligned} \text{trait } T[\overline{\square} \ P <: \{\overline{\square}\}] \text{ end} \in \Delta \\ \text{methods}(\Delta, T[\overline{\alpha}]) &= \left\{ \left(T[\overline{\alpha}], [\overline{\alpha/P}] \mu \right) \right\} \\ \text{object } O[\overline{\square} \ P <: \{\overline{\square}\}] \text{ end} \in \Delta \\ \text{methods}(\Delta, O[\overline{\alpha}]) &= \left\{ \left(O[\overline{\alpha}], [\overline{\alpha/P}] \mu \right) \right\} \end{aligned}$$

All visible methods: $\boxed{\text{allVisible}(\Delta, C) = \{(\overline{C}, \overline{M})\}}$

$$\text{allVisible}(\Delta, C) = \bigcup_{C' \in \text{ancestors}(\Delta, C)} \text{methods}(\Delta, C')$$

Immediately extended traits: $\boxed{\text{parents}(\Delta, C) = \{\overline{C}\}}$

$$\begin{aligned} \text{trait } T[\overline{\square} \ P <: \{\overline{\tau}\}] \text{ end} \in \Delta \\ \text{parents}(\Delta, T[\overline{\alpha}]) &= \{[\overline{\alpha/P}]t\} \\ \text{object } O[\overline{\square} \ P <: \{\overline{\tau}\}] \text{ end} \in \Delta \\ \text{parents}(\Delta, O[\overline{\alpha}]) &= \{[\overline{\alpha/P}]t\} \end{aligned}$$

All extended traits: $\boxed{\text{properAncestors}(\Delta, C) = \{\overline{C}\}}$

$$\text{properAncestors}(\Delta, C) = \bigcup_{C' \in \text{parents}(\Delta, C)} \text{ancestors}(\Delta, C')$$

All extended traits plus self: $\boxed{\text{ancestors}(\Delta, C) = \{\overline{C}\}}$

$$\text{ancestors}(\Delta, C) = \{C\} \cup \text{properAncestors}(\Delta, C)$$

Excluded traits: $\boxed{\text{excluded}(\Delta, C) = \{\overline{C}\}}$

$$\frac{\text{trait } T[\overline{\square} \ P <: \{\overline{\tau}\}] \text{ end} \in \Delta}{\text{excluded}(\Delta, T[\overline{\alpha}]) = \{[\overline{\alpha/P}]t\}}$$

Comprised types: $\boxed{\text{comprised}(\Delta, C) = \{\overline{C}\}}$

$$\begin{aligned} \text{trait } T[\overline{\square} \ P <: \{\overline{\tau}\}] \text{ end} \in \Delta \\ \text{comprised}(\Delta, T[\overline{\alpha}]) &= \{\text{Any}\} \\ \text{trait } T[\overline{\square} \ P <: \{\overline{\tau}\}] \text{ end} \in \Delta \\ \text{comprised}(\Delta, T[\overline{\alpha}]) &= \{[\overline{\alpha/P}]c\} \end{aligned}$$

Parameters in type env: $\boxed{\text{parameters}(\Delta) = \{\overline{P}\}}$

$$\begin{aligned} \text{parameters}(\Delta) &= \bigcup_{\delta \in \Delta} \text{oneParam}(\delta) \\ \text{oneParam}(\{\overline{\square} <: V \ P <: \{\overline{\square}\}\}) &= \{P\} \\ \text{oneParam}(\text{trait } \overline{\square} \text{ end}) &= \{\} \\ \text{oneParam}(\text{object } \overline{\square} \text{ end}) &= \{\} \end{aligned}$$

Trait/object names in env: $\boxed{\text{typeName}(\Delta) = \{\overline{Id}\}}$

$$\begin{aligned} \text{typeName}(\Delta) &= \bigcup \{ \text{oneName}(\delta) \mid \delta \in \Delta \} \\ \text{oneName}(\{\overline{\square} <: V \ P <: \{\overline{\square}\}\}) &= \{\} \\ \text{oneName}(\text{trait } T[\overline{\square}] \text{ end}) &= \{T\} \\ \text{oneName}(\text{object } O[\overline{\square}] \text{ end}) &= \{O\} \end{aligned}$$

Trait/object names in types: $\boxed{\text{typeName}(\{\overline{C}\}) = \{\overline{Id}\}}$

$$\text{typeName}(\{\overline{C}\}) = \{\text{name}(\overline{C})\}$$

Sorting of function or method declarations:

$$\begin{aligned} \frac{d' \in \{\overline{d}\}}{\forall d'' \in \{\overline{d}\}, d''' \in \{\overline{d}\}. (d'' \text{ and } d''' \text{ ordered in } d' \text{ by } \text{lteq})} \\ \text{topologicalSort}(\text{lteq}, \{\overline{d}\}) = \langle \overline{d} \rangle \quad [\text{TOPOLOGICAL-SORT}] \\ \frac{\text{lteq}(d'', d''') \quad \neg(\text{lteq}(d''', d''))}{\langle d' \rangle = \langle \overline{\square}, d'', \overline{\square}, d''', \overline{\square} \rangle} \quad [\text{ORDERED-BEFORE}] \\ \frac{\neg(\text{lteq}(d'', d''')) \quad \text{lteq}(d''', d'')}{\langle d' \rangle = \langle \overline{\square}, d''', \overline{\square}, d'', \overline{\square} \rangle} \quad [\text{ORDERED-AFTER}] \\ \frac{\text{lteq}(d'', d''') = \text{lteq}(d''', d'')}{d'' \text{ and } d''' \text{ ordered in } d' \text{ by } \text{lteq}} \quad [\text{ORDERED-NEITHER}] \end{aligned}$$

Figure 8. Miscellaneous Judgments

abstract function (the tuple of argument types, with any type parameters of the function existentially quantified [3]) must be a subtype of the union of the domains of the concrete definitions. The function *noSelfDom* is used to compute these domain types; the wrinkle is that the same rules are used to check methods as well as functions, and for methods, the type of the trait of object within which the method is declared is *not* considered for the purposes of this particular test.

The rule [T-PROGRAM] furthermore ensure that the function declarations obey the No Duplicates Rule, the Meet Rule, and the Return Type Rule [3]:

- The domain types of two distinct function declarations with the same name may not be identical unless one is abstract and the other is not.
- If two function declarations have the same name, then either (i) their domain types exclude each other (are disjoint), (ii) the domain type of one is a subtype of the domain type of the other, or (iii) there is a third function declaration with the same name whose domain type is a subtype of the domain types of the other two.
- If two function declarations have the same name and the domain type of one is a subtype of the domain type of the other, then intuitively, to ensure type soundness, the return type of the first must be a subtype of the return type of the other—but the details get rather technical when type parameters are involved.

The inference rules for enforcing these relationships appear later in Figure 17. It is worth noting here, however, that the same rules are used for checking both functions and methods, and when methods are considered, the function *dom* (defined in Figure 10) is used to compute domain types, so that the type of the trait or object within which the method is declared *is* considered for the purposes of these other tests.

Figure 8 gives rules for miscellaneous subsidiary judgments used in Figure 6 and elsewhere.

Each of the rules [D-TRAIT], [D-OBJECT], [D-FUNCTION], and [D-METHOD] has two versions, one with all boxed material included and one with all boxed material omitted.

The rule [D-TRAIT] verifies that the names of its type parameters are distinct; that its method definitions are distinct; that all types mentioned are well-formed; that a trait does not extend, comprise, or exclude an instance of itself; that comprised types are subtypes of the comprising trait; that there are no cycles in the “extends” relationship; and that its methods (both those declared within it and those inherited from traits that it extends) obey the No Duplicates Rule, the Meet Rule, and the Return Type Rule. When verifying the well-formedness of methods declared within it, it provides a variable-type environment that has an entry for the special keyword `self`, and it provides a list of its type parameters so that the check for well-formedness of the method can ensure

that the names of the method’s type parameters are distinct from the names of the trait’s type parameters.

We introduce the Ancestors Meet Rule (see Figure 17), which is used by [D-TRAIT] to check the proper ancestors of the declared trait: if a trait extends (directly or indirectly) two different instances $T[\bar{\alpha}]$ and $T[\bar{\eta}]$ of the same trait T , then it must extend some instance $T[\bar{\gamma}]$ of T (which could be $T[\bar{\alpha}]$, $T[\bar{\eta}]$, or a third instance) such that $T[\bar{\gamma}]$ is a subtype of $T[\bar{\alpha}]$ and also a subtype of $T[\bar{\eta}]$. A consequence is that if an object value belongs to one or more instances of some trait T , there is a *unique minimal instance* of T to which it belongs. This is important because being able to identify this minimal instance quickly is essential to the efficient dynamic dispatch algorithms we present later.

The rule [D-OBJECT] is similar in many ways to rule [D-TRAIT]; it verifies that the names of its type parameters are distinct; that its method definitions are distinct; that all types mentioned are well-formed; and that its methods (both those declared within it and those inherited from traits that it extends) obey the No Duplicates Rule, the Meet Rule, and the Return Type Rule. In addition, it verifies that its field names are distinct and that all abstract declarations it inherits are covered by concrete implementations. When verifying the well-formedness of methods declared within it, it provides a variable-type environment that has entries for all its field names and for the special keyword `self`, and it provides a list of its type parameters so that the check for well-formedness of the method can ensure that the names of the method’s type parameters are distinct from the names of the object’s type parameters.

The rule [D-FUNCTION] verifies that the names of its type parameters are distinct; that the names of its value parameters are distinct; that all types mentioned are well-formed; and that its body expression (if present) is well-formed and has a static type that is a subtype of the declared return type ω . Of particular interest is the use of the $\multimap$ construction to define many type environments $\Delta'_1, \Delta'_2, \dots$; what is going on here is that in Welterweight Fortress the type parameters of functions and methods (but not traits and objects) have a scope that extends to the right of their point of declaration but not to the left, so the upper bounds $\{\bar{\xi}\}$ for a type parameter P cannot refer to P or to any type parameter declared to the right of P , but only to type parameters declared to the left of P . This is a change to the Fortress language relative to Fortress 1.0 [1]; this change does not seem to inhibit expressiveness unduly, and introducing this restriction is also essential to the efficient dynamic dispatch algorithms we present later.

The rule [D-METHOD] is similar to rule [D-FUNCTION], but also verifies that the names of its type parameters are distinct one another and from the names of the type parameters of the containing object or trait. One additional wrinkle is that lower bounds $\{\bar{\zeta}\}$ are checked using the type environment Δ rather than one of the instances of Δ' ; the effect is that type parameters of a method may not appear in any

$E ::= \square$	evaluation context	$g ::= X$	ground types	$v ::= O[\bar{g}](\bar{v})$	object instance
$(\bar{e} E \bar{e})$		$(\bar{g})$		$(\bar{v})$	tuple value
$\pi_i(E)$		$(g \rightarrow g)$		$((\bar{x}:\bar{g}):g \Rightarrow e)$	function value
$E\mathbb{Q}(\bar{e})$		Any		$X ::= O[\bar{g}]$	ground constructed types
$e\mathbb{Q}(\bar{e} E \bar{e})$		Object		$T[\bar{g}]$	
$O[\bar{\tau}](\bar{e} E \bar{e})$		$(g \cup g)$		$s ::= g$	signature types
$f(\bar{e} E \bar{e})$		$(g \cap g)$		p	
$E.m(\bar{e})$		$k ::= O[\bar{g}]$	ilks	$p ::= P$	parametric signature types
$e.m(\bar{e} E \bar{e})$		$(\bar{g})$		$O[\bar{g}, p, \bar{s}]$	
$(E \text{ match } x:\tau \Rightarrow e \text{ else } e)$		$\langle g \rightarrow g \rangle$		$T[\bar{g}, p, \bar{s}]$	
$R ::= \pi_i(v)$	redex	Ilk of a value: $\boxed{ilk(v) = k}$		$(\bar{g}, p, \bar{s})$	
(v)		$ilk(O[\bar{g}](\bar{v})) = O[\bar{g}]$		$(p \rightarrow s)$	
$v\mathbb{Q}(\bar{v})$		$ilk((\bar{v})) = (\bar{ilk}(\bar{v}))$		$(g \rightarrow p)$	
$f(\bar{v})$		$ilk(((\bar{x}:\bar{g}):g' \Rightarrow e)) = \langle (\bar{g}) \rightarrow g' \rangle$			
$v.m(\bar{v})$					
$(v \text{ match } x:\tau \Rightarrow e \text{ else } e)$					

Figure 9. Evaluation Contexts, Values, Redexes, Ilks, and Ground Types

$ \begin{array}{lll} msav(\Delta, \Phi, n, (\bar{\chi})) = \{D\} & mostspecific(\Delta, \{\bar{D}\}) = \{D\} & moreSpecific(\Delta)(d, d') \\ applicable(\Delta, \{\bar{d}\}, (\bar{\chi})) = \{\bar{D}\} & dom(d) = \Xi & arrow(d) = \Upsilon \\ instantiate(d, \sigma) = D & visible(\Delta, \Phi, n) = \{\bar{d}\} & firstApplicable(\Delta, \langle \bar{d} \rangle, (\bar{\chi})) = \{D\} \end{array} $	
$msav(\Delta, \Phi, n, (\bar{\chi})) = mostspecific(\Delta, applicable(\Delta, visible(\Delta, \Phi, n), (\bar{\chi})))$	[MSAV-STANDARD]
$mostspecific(\Delta, \{\bar{D}\}) = \left\{ D' \mid D' \in \{\bar{D}\}, \bigwedge_{D'' \in \{\bar{D}\}} moreSpecific(\Delta)(D', D'') \right\}$	[MOST-SPECIFIC]
$moreSpecific(\Delta)(d, d') = \Delta \vdash dom(d) \sqsubseteq dom(d')$	[MORE-SPECIFIC]
$applicable(\Delta, \{\bar{d}\}, (\bar{\chi})) = \{ instantiate(d', \sigma) \mid d' \in \{\bar{d}\}, \Delta \vdash \exists \llbracket \bar{\chi} \rrbracket \sqsubseteq dom(d') \text{ using } \sigma \}$	[APPLICABLE]
$dom(f \llbracket P <: \{\bar{\xi}\} \rrbracket (\bar{x}:\bar{\tau}) : \omega = e) = \exists \llbracket \{\bar{\xi}\} <: P <: \{\bar{\xi}\} \rrbracket (\bar{\tau})$	[DOM-FUNCTION]
$dom((C, m \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket (\bar{x}:\bar{\alpha}) : \rho = e)) = \exists \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket (C, \bar{\alpha})$	[DOM-METHOD]
$arrow(f \llbracket P <: \{\bar{\xi}\} \rrbracket (\bar{x}:\bar{\tau}) : \omega = e) = \forall \llbracket \{\bar{\xi}\} <: P <: \{\bar{\xi}\} \rrbracket ((\bar{\tau}) \rightarrow \omega)$	[ARROW-FUNCTION]
$arrow((C, m \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket (\bar{x}:\bar{\alpha}) : \rho = e)) = \forall \llbracket \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \rrbracket ((C, \bar{\alpha}) \rightarrow \rho)$	[ARROW-METHOD]
$instantiate(f \llbracket \bar{\tau} \rrbracket (\bar{x}:\bar{\tau}) : \omega = e, \sigma) = f \llbracket \bar{\tau} \rrbracket (\bar{x}:\bar{\sigma}\bar{\tau}) : (\sigma\omega) = \sigma e$	[INstantiate-FUNCTION]
$instantiate((C, m \llbracket \bar{\tau} \rrbracket (\bar{x}:\bar{\alpha}) : \rho = e), \sigma) = (C, m \llbracket \bar{\tau} \rrbracket (\bar{x}:\bar{\sigma}\bar{\alpha}) : (\sigma\rho) = \sigma e)$	[INstantiate-METHOD]
$visible(\Delta, \Phi, f) = \{ \varphi \mid \varphi \in \Phi, name(\varphi) = f \}$	[VISIBLE-FUNCTIONS]
$visible(\Delta, \Phi, (C, m)) = \bigcup_{C' \in ancestors(\Delta, C')} \{ (C'', M) \mid (C'', M) \in methods(\Delta, C'), name(M) = m \}$	[VISIBLE-METHODS]
$msav(\Delta, \Phi, n, (\bar{\chi})) = firstApplicable(\Delta, topologicalSort(moreSpecific(\Delta), visible(\Delta, \Phi, n)), (\bar{\chi}))$	[MSAV-ALTERNATE]
$ \frac{\langle \bar{d} \rangle = \langle \bar{d}', d'', \bar{\tau} \rangle \quad applicable(\Delta, d', (\bar{\chi})) = \{ \} \quad applicable(\Delta, d'', (\bar{\chi})) = \{ D \}}{firstApplicable(\Delta, \langle \bar{d} \rangle, (\bar{\chi})) = \{ D \}} $	[FIRST-APPLICABLE]

Figure 10. Determining the most specific applicable visible function or method

Program reduction: $\boxed{\vdash \Pi \longrightarrow^* e}$

$$\frac{\Pi = \bar{\psi}, \bar{\varphi}, e \quad \{\bar{\psi}\}; \{\bar{\varphi}\} \vdash e \longrightarrow^* e'}{\vdash \Pi \longrightarrow^* e'} \quad [\text{R-PROGRAM}]$$

Expression reduction: $\boxed{\Delta; \Phi \vdash E[R] \longrightarrow^* E[e]}$

$$\Delta; \Phi \vdash e \longrightarrow^* e \quad [\text{R-BASE}]$$

$$\frac{\Delta; \Phi \vdash e \longrightarrow^* e' \quad \Delta; \Phi \vdash e' \longrightarrow e''}{\Delta; \Phi \vdash e \longrightarrow^* e''} \quad [\text{R-INDUCTION}]$$

Expression reduction step: $\boxed{\Delta; \Phi \vdash E[R] \longrightarrow E[e]}$

$$\Delta; \Phi \vdash \underline{E}[\pi(\bar{v})] \longrightarrow \underline{E}[v] \quad [\text{R-PROJECT-TUPLE}]$$

$$\Delta; \Phi \vdash (v) \longrightarrow v \quad [\text{R-SINGLETON-TUPLE}]$$

$$\Delta; \Phi \vdash E[(\bar{x}:-) _ \Rightarrow e] \mathcal{Q}(\bar{v}) \longrightarrow E[\bar{v}/x]e \quad [\text{R-APPLY}]$$

$$\frac{msav(\Delta, \Phi, f, (\bar{ilk}(\bar{v}))) = \{f \llbracket \bar{x}:- _ = e \rrbracket\}}{\Delta; \Phi \vdash E[f(\bar{v})] \longrightarrow E[\bar{v}/x]e} \quad [\text{R-FUNCTION}]$$

$$\frac{\text{object } O[\bar{x}:-] _ \text{end} \in \Delta \quad v'' = O[\bar{\gamma}](\bar{v}) \quad msav(\Delta, \Phi, (O[\bar{\gamma}], m), (\bar{ilk}(\bar{v}))) = \{m \llbracket \bar{x}:- _ = e \rrbracket\}}{\Delta; \Phi \vdash E[v''.m(\bar{v}')] \longrightarrow E[\bar{v}''/\text{self}][\bar{v}/x][\bar{v}'/x']e} \quad [\text{R-METHOD}]$$

$$\frac{\Delta \vdash \bar{ilk}(\bar{v}) <: \tau}{\Delta; \Phi \vdash E[(v \text{ match } x: \tau \Rightarrow e \text{ else } e')] \longrightarrow E[\bar{v}/x]e} \quad [\text{R-MATCH-SUCCEEDS}]$$

$$\frac{\Delta \vdash \bar{ilk}(\bar{v}) \not<: \tau}{\Delta; \Phi \vdash E[(v \text{ match } x: \tau \Rightarrow e \text{ else } e')] \longrightarrow E[e']} \quad [\text{R-MATCH-FAILS}]$$

Figure 11. Dynamic Semantics: Evaluation Rules

lower bound ζ . (However, type parameters of the enclosing trait *are* allowed to appear in lower bounds ζ .) This is another change to the Fortress language relative to Fortress 1.0 [1]; this change also does not seem to inhibit expressiveness unduly, and is essential to the efficient dynamic dispatch algorithms we present later.

The rules for typing and well-formedness depend on judgments about subtyping and type exclusion, which are addressed later in Section 6.

5. Dynamic Evaluation Semantics

Figure 9 gives conventional grammars for redexes (a subset of expressions that are subject to evaluation reduction), values (expressions that contain no redexes), and evaluation contexts. It also defines how to compute the ilk of a value. The notation $\langle \alpha \rightarrow \omega \rangle$ is the type of functions that produce a value of type ω when given an argument of type α and furthermore go wrong (“get stuck”) when given an argument

Constraint Grammar **Constraint negation:** $\boxed{\neg C = C}$

$C ::=$	$T <: T'$	$\neg(T <: T') =$	$T \not<: T'$
$ $	$T \diamond T'$	$\neg(T \diamond T') =$	$T \not\diamond T'$
$ $	$T \not<: T'$	$\neg(T \not<: T') =$	$T <: T'$
$ $	$T \not\diamond T'$	$\neg(T \not\diamond T') =$	$T \diamond T'$
$ $	$\mathcal{C} \wedge \mathcal{C}$	$\neg(\mathcal{C} \wedge \mathcal{C}) =$	$\neg(\mathcal{C}) \vee \neg(\mathcal{C})$
$ $	$\mathcal{C} \vee \mathcal{C}$	$\neg(\mathcal{C} \vee \mathcal{C}) =$	$\neg(\mathcal{C}) \wedge \neg(\mathcal{C})$
$ $	false	$\neg(\text{false}) =$	true
$ $	true	$\neg(\text{true}) =$	false

Convert type env to constraints: $\boxed{toConstraint(\Delta) = C}$

Convert constraints to type env: $\boxed{toBounds(C) = \Delta}$

Solve constraints: $\boxed{\Delta \vdash unify(C) = (\sigma, C)}$

Figure 12. Constraints. The $\wedge$, $\vee$, and $\neg$ operators form a Boolean algebra; *unify* and *toBounds* are partial functions.

that is not of type α . Ilks (the possible “run-time types” of values), ground types (types that contain no type parameters), signature types (types that can contain type parameters, but not within an operand of $\cap$ or $\cup$), and parametric signature types (signature types that mention at least one type parameter) are also defined.

In Welterweight Fortress there are three kinds of values: objects, closures, and tuples. Every object belongs to at least one constructed type, every closure belongs to at least one arrow type, and every tuple belongs to at least one tuple type. No value belongs to Bottom.

Although a value may belong to more than one type, for every value v there is a unique type $\bar{ilk}(v)$ (the *ilk* of the value) such that for every type α , if v belongs to α then $\bar{ilk}(v)$ is a subtype of α . (This notion of *ilk* corresponds to what is sometimes called the “class” or “run-time type” of the value. We prefer the term “ilk” to “run-time type” because the notion—and usefulness—of the most specific type to which a value belongs is not confined to run time.)

The implementation significance of ilks is that it is possible to select the dynamically most specific applicable function or method from an overload set using only the ilks of the argument values; no other information about the arguments is needed.

Figure 10 gives the rules for determining which is the most specific applicable function or method for a given tuple of argument types. It also defines a function *arrow* that is related to *dom* and is needed for the Return Type Rule in Figure 17. Figure 11 gives rules for reducing an expression to a value. The rules [R-MATCH-SUCCEEDS] and [R-MATCH-FAILS] test the ilk of a value to decide which of two expressions to evaluate. The rules [R-FUNCTION] and [R-METHOD] are the dynamic counterparts to rules [T-FUNCTION] and [T-METHOD] of Figure 5: they also use the function *msav* to find the most specific applicable visible

function or method for that invocation. However, the last argument to *msav* in rules [R-FUNCTION] and [R-METHOD] is not a tuple of the static types of the argument expressions, but rather a tuple of the *ilks* of the argument *values*; therefore these uses of *msav* identifies the *dynamically* most specific applicable visible function or method.

For the propositions that follow, assume a program $\Pi = \overline{\psi}, \overline{\varphi}, e$ is given, let $\Delta = \{\overline{\psi}\}$, and assume Π is well-formed (that is, $\vdash \Pi : g$ for some g).

Proposition 1 Whenever rule [T-FUNCTION] or [T-METHOD] is used, *msav* always identifies exactly one function or method (thanks in large part to the Meet Rule).

Proposition 2 Whenever rule [R-FUNCTION] is used, *msav* always identifies exactly one function (thanks in large part to the Meet Rule), and the function identified is more specific than (or the same as) the function identified by rule [T-FUNCTION] for that call site.

If our type system is sound, then the following propositions should also hold:

Proposition 3 If for any expression e in the program we have $\Delta; \Phi; \Gamma \vdash e : \alpha$, then every value computed by the expression at run time according to rules of Figure 11 will belong to type α .

Proposition 4 Whenever a closure whose ilk is $\langle \alpha \rightarrow \omega \rangle$ is applied to an argument value by rule [R-APPLY], the argument value will belong to type α .

Proposition 5 Whenever a function or method is invoked, the argument values will belong to the types of the corresponding parameters.

The two big questions are of course these:

Proposition 6 (Termination) Reduction of a well-formed program never gets “stuck” (unable to take a reduction step) before a value is produced.

Proposition 7 (Soundness) If $\vdash \Pi : g$ and $\vdash \Pi \longrightarrow^* v$ (note that v refers to a value), then $\Delta \vdash ilk(v) <: g$.

We have not yet proved any of these propositions rigorously. Welterweight Fortress is based on the algorithms now in use in our prototype Fortress compiler; these algorithms are constantly vetted against a test suite as we continue to develop the code and have been in use for months (and in some cases years). Nevertheless, errors may have crept in as we have transcribed and codified these algorithms into the inference rules presented here. We do have experience in mechanizing proofs of properties of a smaller model, Featherweight Fortress [8], using Coq. We expect that mechanizing proofs of the preceding propositions is feasible but will require many months of effort.

6. Subtyping and Type Exclusion

Our subtyping judgments are constraint-based and rely in the general case on solving systems of constraints on type parameters. Figures 12, 13, 14, 15, and 16 are adapted and improved from our previously published work [3], and Figure 17 has been newly constructed based on material in the text of that paper. We summarize the content of these figures briefly, using text that is in large part adapted and condensed from that earlier paper. A notable improvement appears at the end of this section.

Figure 12 gives a grammar for type constraints. A primitive constraint is either *positive* ($T <: T'$ specifies that a T is a subtype of T' , and $T \diamond T'$ specifies that T excludes T') or *negative* ($T \not<: T'$ specifies that a T is not a subtype of T' , and $T \not\diamond T'$ specifies that T and T' must have some value in common). A conjunction constraint $C_1 \wedge C_2$ is satisfied exactly when both C_1 and C_2 are satisfied, and a disjunction constraint $C_1 \vee C_2$ is satisfied exactly when one or both of C_1 and C_2 are satisfied. The constraint false is never satisfied, and the constraint true is always satisfied. The equivalence constraint $T \equiv T'$ is derived as $T <: T' \wedge T' <: T$.

Following Smith and Cartwright [14], in practice we normalize all constraint formulas into disjunctive normal form and simplify using absorption laws. The negation $\neg C$ of a constraint C has a standard de Morgan interpretation. Each type environment Δ naturally describes a constraint on the type parameters *parameters*(Δ), which we denote *toConstraint*(Δ). This conversion has a partial inverse *toBound*(C) that is defined whenever C can be written as a conjunction of constraints of the form $P <: \alpha$ (if C has multiple conjuncts of this form for a single P , then the resulting environment contains multiple upper bounds for P). The judgment $\Delta \vdash \text{unify}(C) = (\sigma, C')$ solves a set of constraints on inference variables using unification, producing a substitution σ for the inference variables and a reduced set of constraints C' on type parameters that is free of inference variables. (These are used in the definition of existential reduction in Figure 16.)

Figure 13 gives rules for computing a set of constraints that, if satisfied, will guarantee a (reflexive) subtype relationship $<:$ between two types. (Conversely, if the constraints are not satisfiable, then a subtype relationship does *not* hold between the two types.) Figure 14 gives rules for computing a set of constraints that, if satisfied, will guarantee an exclusion relationship $\diamond$ between two types. (Conversely, if the constraints are not satisfiable, then an exclusion relationship does *not* hold between the two types.)

As explained in [3], we use negative judgments $\Delta \vdash T \not<: T \Leftarrow C$ and $\Delta \vdash T \not\diamond T \Leftarrow C$ to determine constraints under which the negations hold. To save space, rather than completely defining negative judgments explicitly, we describe how to derive most of their inference rules from their positive counterparts according to de Morgan’s laws. The rules for bound variables are given explicitly in

Subtyping Rules That Generate Constraints: $\boxed{\Delta \vdash \alpha <: \alpha \Leftarrow \mathcal{C} \quad \Delta \vdash \alpha \not<: \alpha \Leftarrow \mathcal{C}}$ (see text)

Logical rules

$$\begin{array}{l}
\Delta \vdash \alpha <: \alpha \Leftarrow \text{true} \\
\Delta \vdash \text{Bottom} <: \alpha \Leftarrow \text{true} \\
\Delta \vdash c <: \text{Bottom} \Leftarrow \text{false} \\
\Delta \vdash \text{Any} <: \text{Object} \Leftarrow \text{false} \\
\Delta \vdash \text{Any} <: \text{Bottom} \Leftarrow \text{false} \\
\Delta \vdash \text{Object} <: \text{Object} \Leftarrow \text{true} \\
\Delta \vdash \text{Object} <: \text{Bottom} \Leftarrow \text{false} \\
\Delta \vdash \text{Object} <: c \Leftarrow \text{false} \\
\Delta \vdash c <: \text{Object} \Leftarrow \text{true} \\
\Delta \vdash \text{Any} <: c \Leftarrow \text{false} \\
\Delta \vdash \alpha <: \text{Any} \Leftarrow \text{true} \\
\frac{\Delta \vdash \alpha <: \eta \Leftarrow \mathcal{C} \quad \Delta \vdash \alpha <: \chi \Leftarrow \mathcal{C}'}{\Delta \vdash \alpha <: (\eta \cap \chi) \Leftarrow \mathcal{C} \wedge \mathcal{C}'} \\
\frac{\Delta \vdash \alpha <: \chi \Leftarrow \mathcal{C} \quad \Delta \vdash \eta <: \chi \Leftarrow \mathcal{C}' \quad \Delta \vdash \alpha \diamond \eta \Leftarrow \mathcal{C}''}{\Delta \vdash (\alpha \cap \eta) <: \chi \Leftarrow \mathcal{C} \vee \mathcal{C}' \vee \mathcal{C}''} \\
\frac{\Delta \vdash \alpha <: \eta \Leftarrow \mathcal{C} \quad \Delta \vdash \alpha <: \chi \Leftarrow \mathcal{C}'}{\Delta \vdash \alpha <: (\eta \cup \chi) \Leftarrow \mathcal{C} \vee \mathcal{C}'} \\
\frac{\Delta \vdash \alpha <: \chi \Leftarrow \mathcal{C} \quad \Delta \vdash \eta <: \chi \Leftarrow \mathcal{C}'}{\Delta \vdash (\alpha \cup \eta) <: \chi \Leftarrow \mathcal{C} \wedge \mathcal{C}'}
\end{array}$$

Variance test: $\boxed{\Delta \vdash \alpha V \alpha \Leftarrow \mathcal{C}}$

$$\begin{array}{l}
\frac{\Delta \vdash \alpha <: \eta \Leftarrow \mathcal{C}}{\Delta \vdash \alpha \text{ covariant } \eta \Leftarrow \mathcal{C}} \\
\frac{\Delta \vdash \eta <: \alpha \Leftarrow \mathcal{C}}{\Delta \vdash \alpha \text{ contravariant } \eta \Leftarrow \mathcal{C}} \\
\frac{\Delta \vdash \alpha \equiv \eta \Leftarrow \mathcal{C}}{\Delta \vdash \alpha \text{ invariant } \eta \Leftarrow \mathcal{C}}
\end{array}$$

Inference Variables

$$\begin{array}{l}
\frac{I \notin \text{parameters}(\Delta)}{\Delta \vdash I <: I \Leftarrow \text{true}} \\
\frac{I \notin \text{parameters}(\Delta)}{\Delta \vdash I <: \alpha \Leftarrow I <: \alpha} \\
\frac{I \notin \text{parameters}(\Delta)}{\Delta \vdash \alpha <: I \Leftarrow \alpha <: I}
\end{array}$$

Bound Variables

$$\begin{array}{l}
\Delta \vdash P <: P \Leftarrow \text{true} \\
\frac{\{\bar{_}\} <: V P <: \{\bar{\xi}\} \in \Delta}{\Delta \vdash \bar{P} <: \xi \Leftarrow \text{true}} \\
\frac{\{\bar{_}\} <: V P <: \{\bar{_}\} \in \Delta}{\Delta \vdash \bar{\zeta} <: \bar{P} \Leftarrow \text{true}} \\
\frac{\Delta \vdash \alpha <: \text{Bottom} \Leftarrow \mathcal{C}}{\Delta \vdash \alpha <: P \Leftarrow \mathcal{C}} \\
\frac{\{\bar{\gamma}\} <: V P <: \{\bar{\eta}\} \in \Delta \quad \Delta \vdash \bar{\alpha} \not<: \gamma \Leftarrow \mathcal{C} \quad \Delta \vdash \bar{\alpha} \diamond \eta \Leftarrow \mathcal{C}'}{\Delta \vdash \alpha \not<: P \Leftarrow \bigvee \{\bar{\mathcal{C}}, \bar{\mathcal{C}}'\}} \\
\frac{\{\bar{_}\} <: V P <: \{\bar{\eta}\} \in \Delta \quad \Delta \vdash \bar{\alpha} \not<: \eta \Leftarrow \mathcal{C}}{\Delta \vdash P \not<: \alpha \Leftarrow \bigvee \{\bar{\mathcal{C}}\}}
\end{array}$$

Structural rules

$$\begin{array}{l}
\frac{\#(\bar{\alpha}) = \#(\bar{\eta}) \quad \Delta \vdash \bar{\alpha} <: \eta \Leftarrow \mathcal{C}}{\Delta \vdash \bar{\alpha} <: \text{Bottom} \Leftarrow \mathcal{C}'} \\
\frac{\Delta \vdash (\bar{\alpha}) <: (\bar{\eta}) \Leftarrow (\bigwedge \{\bar{\mathcal{C}}\}) \vee (\bigvee \{\bar{\mathcal{C}}'\})}{\#(\bar{\alpha}) \neq 1 \quad \Delta \vdash \bar{\alpha} <: \text{Bottom} \Leftarrow \mathcal{C}} \\
\frac{\Delta \vdash (\bar{\alpha}) <: \text{Bottom} \Leftarrow \bigvee \{\bar{\mathcal{C}}\}}{\Delta \vdash T[\bar{\alpha}] \text{ ok} \quad \Delta \vdash \bar{\alpha} <: \text{Bottom} \Leftarrow \mathcal{C}} \\
\frac{\Delta \vdash T[\bar{\alpha}] <: \text{Bottom} \Leftarrow \bigvee \{\bar{\mathcal{C}}\}}{\Delta \vdash \alpha' <: \alpha \Leftarrow \mathcal{C} \quad \Delta \vdash \rho <: \rho' \Leftarrow \mathcal{C}'} \\
\frac{\Delta \vdash (\alpha \rightarrow \rho) <: (\alpha' \rightarrow \rho') \Leftarrow \mathcal{C} \wedge \mathcal{C}'}{\#(\bar{\eta}) \neq 1} \\
\frac{\#(\bar{\eta}) \neq 1}{\Delta \vdash (\alpha \rightarrow \rho) <: (\bar{\eta}) \Leftarrow \text{false}} \\
\frac{\#(\bar{\alpha}) \neq 1}{\Delta \vdash c <: (\bar{\alpha}) \Leftarrow \text{false}} \\
\Delta \vdash (\alpha \rightarrow \rho) <: c \Leftarrow \text{false} \\
\Delta \vdash c <: (\alpha \rightarrow \rho) \Leftarrow \text{false} \\
\Delta \vdash (\alpha \rightarrow \rho) <: \text{Bottom} \Leftarrow \text{false} \\
\Delta \vdash \text{Any} <: (\alpha \rightarrow \omega) \Leftarrow \text{false} \\
\frac{\#(\bar{\alpha}) \neq 1}{\Delta \vdash \text{Object} <: (\bar{\alpha}) \Leftarrow \text{false}} \\
\frac{\#(\bar{\alpha}) \neq 1}{\Delta \vdash \text{Any} <: (\bar{\alpha}) \Leftarrow \text{false}} \\
\Delta \vdash \text{Object} <: (\alpha \rightarrow \rho) \Leftarrow \text{false} \\
\Delta \vdash (\alpha \rightarrow \rho) <: \text{Object} \Leftarrow \text{false}
\end{array}$$

Constructed types

$$\begin{array}{l}
\frac{\text{distinct}(T, T') \quad \Delta \vdash T[\bar{\alpha}] \text{ ok} \quad \Delta \vdash T'[\bar{\eta}] \text{ ok} \quad \{\bar{\chi}\} = \text{parents}(\Delta, T[\bar{\alpha}]) \quad \Delta \vdash \bar{\chi} <: \overline{T'[\bar{\eta}]} \Leftarrow \mathcal{C}}{\Delta \vdash T[\bar{\alpha}] <: T'[\bar{\eta}] \Leftarrow \bigvee \{\bar{\mathcal{C}}\}} \text{ [SUB-T-PARENTS]} \\
\frac{\Delta \vdash O[\bar{\alpha}] \text{ ok} \quad \Delta \vdash T'[\bar{\eta}] \text{ ok} \quad \{\bar{\chi}\} = \text{parents}(\Delta, O[\bar{\alpha}]) \quad \Delta \vdash \bar{\chi} <: \overline{T'[\bar{\eta}]} \Leftarrow \mathcal{C}}{\Delta \vdash O[\bar{\alpha}] <: T'[\bar{\eta}] \Leftarrow \bigvee \{\bar{\mathcal{C}}\}} \text{ [SUB-O-PARENTS]} \\
\frac{\text{trait } T[\bar{V} P <: \{\bar{_}\}] \text{ -- end} \in \{\Delta\} \quad \Delta \vdash T[\bar{\alpha}] \text{ ok} \quad \Delta \vdash T[\bar{\eta}] \text{ ok} \quad \Delta \vdash \bar{\alpha} V \eta \Leftarrow \mathcal{C}}{\Delta \vdash T[\bar{\alpha}] <: T[\bar{\eta}] \Leftarrow \bigwedge \{\bar{\mathcal{C}}\}} \text{ [SUB-VARIANCE]} \\
\frac{\Delta \vdash T[\bar{\alpha}] \text{ ok} \quad \{\bar{\chi}\} = \text{comprised}(\Delta, T[\bar{\alpha}]) \quad \Delta \vdash \bar{\chi} <: \bar{\eta} \Leftarrow \mathcal{C}}{\Delta \vdash T[\bar{\alpha}] <: \eta \Leftarrow \bigwedge \{\bar{\mathcal{C}}\}} \text{ [SUB-COMPRISES]}
\end{array}$$

Figure 13. Algorithm for generating subtyping constraints. (Adapted and extended from [3].) Apply the first rule that matches.

Exclusion Rules That Generate Constraints: $\boxed{\Delta \vdash \alpha \diamond \alpha \Leftarrow \mathcal{C} \quad \Delta \vdash \alpha \not\Leftarrow \alpha \Leftarrow \mathcal{C}}$ (see text)

Logical rules

$$\Delta \vdash \text{Bottom} \diamond \alpha \Leftarrow \text{true}$$

$$\frac{\Delta \vdash \alpha <: \text{Bottom} \Leftarrow \mathcal{C}}{\Delta \vdash \text{Any} \diamond \alpha \Leftarrow \mathcal{C}}$$

$$\frac{\#(\bar{\alpha}) \neq 1}{\Delta \vdash (\bar{\alpha}) \diamond \text{Object} \Leftarrow \text{true}}$$

$$\Delta \vdash (\alpha \rightarrow \rho) \diamond \text{Object} \Leftarrow \text{true}$$

$$\frac{\begin{array}{c} \Delta \vdash \alpha \diamond \chi \Leftarrow \mathcal{C} \quad \Delta \vdash \eta \diamond \chi \Leftarrow \mathcal{C}' \\ \Delta \vdash (\alpha \cap \eta) <: \text{Bottom} \Leftarrow \mathcal{C}'' \end{array}}{\Delta \vdash (\alpha \cap \eta) \diamond \chi \Leftarrow \mathcal{C} \vee \mathcal{C}' \vee \mathcal{C}''}$$

$$\frac{\Delta \vdash \alpha \diamond \chi \Leftarrow \mathcal{C} \quad \Delta \vdash \eta \diamond \chi \Leftarrow \mathcal{C}'}{\Delta \vdash (\alpha \cup \eta) \diamond \chi \Leftarrow \mathcal{C} \wedge \mathcal{C}'}$$

Inference Variables

$$\frac{I \notin \text{parameters}(\Delta)}{\Delta \vdash I \diamond I \Leftarrow \text{false}}$$

$$\frac{I \notin \text{parameters}(\Delta)}{\Delta \vdash I \diamond \alpha \Leftarrow I \diamond \alpha}$$

Bound Variables

$$\frac{\begin{array}{c} \{\bar{\cdot}\} <: V P <: \{\bar{\xi}\} \in \Delta \\ \Delta \vdash \bar{\xi} \diamond \alpha \Leftarrow \bar{\mathcal{C}} \end{array}}{\Delta \vdash P \diamond \alpha \Leftarrow \bigvee \{\bar{\mathcal{C}}\}}$$

$$\frac{\begin{array}{c} \{\bar{\cdot}\} <: V P <: \{\bar{\cdot}\} \in \Delta \\ \Delta \vdash \alpha <: P \Leftarrow \mathcal{C} \quad \Delta \vdash \alpha \not<: \text{Bottom} \Leftarrow \mathcal{C}' \end{array}}{\Delta \vdash P \not\Leftarrow \alpha \Leftarrow \mathcal{C} \wedge \mathcal{C}'}$$

Structural rules

$$\frac{\#(\bar{\alpha}) = \#(\bar{\eta}) \quad \Delta \vdash \bar{\alpha} \diamond \bar{\eta} \Leftarrow \bar{\mathcal{C}}}{\Delta \vdash (\bar{\alpha}) \diamond (\bar{\eta}) \Leftarrow \bigvee \{\bar{\mathcal{C}}\}}$$

$$\frac{\#(\bar{\alpha}) \neq \#(\bar{\eta})}{\Delta \vdash (\bar{\alpha}) \diamond (\bar{\eta}) \Leftarrow \text{true}}$$

$$\frac{\#(\bar{\eta}) \neq 1}{\Delta \vdash c \diamond (\bar{\eta}) \Leftarrow \text{true}}$$

$$\frac{\#(\bar{\eta}) \neq 1}{\Delta \vdash (\alpha \rightarrow \rho) \diamond (\bar{\eta}) \Leftarrow \text{true}}$$

$$\Delta \vdash (\alpha \rightarrow \rho) \diamond (\alpha' \rightarrow \rho') \Leftarrow \text{false}$$

$$\Delta \vdash c \diamond (\alpha \rightarrow \rho) \Leftarrow \text{true}$$

Constructed types

$$\frac{\begin{array}{c} \Delta \vdash c \diamond_x c' \Leftarrow \mathcal{C}_x \quad \Delta \vdash c \diamond_c c' \Leftarrow \mathcal{C}_c \\ \Delta \vdash c \diamond_o c' \Leftarrow \mathcal{C}_o \quad \Delta \vdash c \diamond_m c' \Leftarrow \mathcal{C}_m \end{array}}{\Delta \vdash c \diamond c' \Leftarrow \mathcal{C}_x \vee \mathcal{C}_c \vee \mathcal{C}_o \vee \mathcal{C}_m}$$

$$\frac{\Delta \vdash c \triangleright_x c' \Leftarrow \mathcal{C} \quad \Delta \vdash c' \triangleright_x c \Leftarrow \mathcal{C}'}{\Delta \vdash c \diamond_x c' \Leftarrow \mathcal{C} \vee \mathcal{C}'}$$

$$\frac{\Delta \vdash c \triangleright_c c' \Leftarrow \mathcal{C} \quad \Delta \vdash c' \triangleright_c c \Leftarrow \mathcal{C}'}{\Delta \vdash c \diamond_c c' \Leftarrow \mathcal{C} \vee \mathcal{C}'}$$

$$\frac{\Delta \vdash c \triangleright_o c' \Leftarrow \mathcal{C} \quad \Delta \vdash c' \triangleright_o c \Leftarrow \mathcal{C}'}{\Delta \vdash c \diamond_o c' \Leftarrow \mathcal{C} \vee \mathcal{C}'}$$

$$\Delta \vdash T[\bar{\alpha}] \triangleright_x T[\bar{\eta}] \Leftarrow \text{false}$$

$$\Delta \vdash T[\bar{\alpha}] \triangleright_c T[\bar{\eta}] \Leftarrow \text{false}$$

$$\Delta \vdash T[\bar{\alpha}] \triangleright_o T[\bar{\eta}] \Leftarrow \text{false}$$

$$\frac{\begin{array}{c} \text{distinct}(T, T') \quad \{\bar{\gamma}\} = \text{ancestors}(\Delta, T[\bar{\alpha}]) \\ \{\bar{\chi}\} = \bigcup \{\text{excluded}(\Delta, \gamma)\} \quad \Delta \vdash \overline{T'[\bar{\eta}]} <: \chi \Leftarrow \bar{\mathcal{C}} \end{array}}{\Delta \vdash T[\bar{\alpha}] \triangleright_x T'[\bar{\eta}] \Leftarrow \bigvee \{\bar{\mathcal{C}}\}}$$

$$\frac{\begin{array}{c} \text{distinct}(T, T') \\ \{\bar{\chi}\} = \text{comprised}(\Delta, T[\bar{\alpha}]) \quad \Delta \vdash \overline{T'[\bar{\eta}]} \diamond \chi \Leftarrow \bar{\mathcal{C}} \end{array}}{\Delta \vdash T[\bar{\alpha}] \triangleright_c T'[\bar{\eta}] \Leftarrow \bigwedge \{\bar{\mathcal{C}}\}}$$

$$\frac{\Delta \vdash O[\bar{\alpha}] \not<: c \Leftarrow \mathcal{C}}{\Delta \vdash O[\bar{\alpha}] \triangleright_o c \Leftarrow \mathcal{C}}$$

$$\frac{\text{distinct}(T, T')}{\Delta \vdash T[\bar{\alpha}] \triangleright_c T'[\bar{\eta}] \Leftarrow \text{false}}$$

$$\frac{\begin{array}{c} \{\bar{\gamma}\} = \text{ancestors}(\Delta, T[\bar{\alpha}]) \quad \{\bar{\chi}\} = \text{ancestors}(\Delta, T'[\bar{\eta}]) \\ \Delta \vdash \bar{\gamma} \diamond_m \bar{\chi} \Leftarrow \bar{\mathcal{C}} \end{array}}{\Delta \vdash T[\bar{\alpha}] \diamond_m T'[\bar{\eta}] \Leftarrow \bigvee \{\bar{\mathcal{C}}\}}$$

$$\frac{\Delta \vdash \bar{\alpha} \not\Leftarrow \bar{\eta} \Leftarrow \bar{\mathcal{C}}}{\Delta \vdash T[\bar{\alpha}] \diamond_m T[\bar{\eta}] \Leftarrow \bigvee \{\bar{\mathcal{C}}\}}$$

$$\frac{\text{distinct}(T, T')}{\Delta \vdash T[\bar{\alpha}] \diamond_m T'[\bar{\eta}] \Leftarrow \text{false}}$$

Figure 14. Algorithm for generating exclusion constraints. (Adapted and extended from [3].) Each rule is symmetric (because $\diamond$ is a symmetric relation), so try it twice, with operands as given and with operands swapped; apply the first one that matches.

Constrained type equivalence: $\boxed{\Delta \vdash \alpha \equiv \alpha \Leftarrow \mathcal{C}}$

$$\frac{\Delta \vdash \alpha <: \eta \Leftarrow \mathcal{C} \quad \Delta \vdash \eta <: \alpha \Leftarrow \mathcal{C}'}{\Delta \vdash \alpha \equiv \eta \Leftarrow \mathcal{C} \wedge \mathcal{C}'}$$

Subtyping: $\boxed{\Delta \vdash \alpha <: \alpha \quad \Delta \vdash \alpha \not<: \alpha}$

$$\frac{\Delta \vdash \alpha <: \eta \Leftarrow \text{true}}{\Delta \vdash \alpha <: \eta} \quad \frac{\Delta \vdash \alpha \not<: \eta \Leftarrow \text{true}}{\Delta \vdash \alpha \not<: \eta}$$

Exclusion: $\boxed{\Delta \vdash \alpha \diamond \alpha \quad \Delta \vdash \alpha \not\diamond \alpha}$

$$\frac{\Delta \vdash \alpha \diamond \eta \Leftarrow \text{true}}{\Delta \vdash \alpha \diamond \eta} \quad \frac{\Delta \vdash \alpha \not\diamond \eta \Leftarrow \text{true}}{\Delta \vdash \alpha \not\diamond \eta}$$

Equivalence: $\boxed{\Delta \vdash \alpha \equiv \alpha \quad \Delta \vdash \alpha \not\equiv \alpha}$

$$\frac{\Delta \vdash \alpha \equiv \eta \Leftarrow \text{true}}{\Delta \vdash \alpha \equiv \eta} \quad \frac{\Delta \vdash \alpha \not\equiv \eta \Leftarrow \text{true}}{\Delta \vdash \alpha \not\equiv \eta}$$

Variance test: $\boxed{\Delta \vdash \alpha V \alpha}$

$$\frac{\Delta \vdash \alpha V \eta \Leftarrow \text{true}}{\Delta \vdash \alpha V \eta}$$

Conditional equivalence: $\boxed{\Delta \vdash \alpha \equiv \alpha \Leftarrow \mathcal{C}}$

$$\frac{\Delta \vdash \alpha <: \eta \Leftarrow \mathcal{C} \quad \Delta \vdash \eta <: \alpha \Leftarrow \mathcal{C}'}{\Delta \vdash \alpha \equiv \eta \Leftarrow \mathcal{C} \wedge \mathcal{C}'}$$

Conditional non-equivalence: $\boxed{\Delta \vdash \alpha \not\equiv \alpha \Leftarrow \mathcal{C}}$

$$\frac{\Delta \vdash \alpha \not<: \eta \Leftarrow \mathcal{C} \quad \Delta \vdash \eta \not<: \alpha \Leftarrow \mathcal{C}'}{\Delta \vdash \alpha \not\equiv \eta \Leftarrow \mathcal{C} \vee \mathcal{C}'}$$

Converse non-equivalence: $\boxed{\Delta \vdash \alpha \not\equiv \alpha \Rightarrow \mathcal{C}}$

$$\frac{\Delta \vdash \alpha \equiv \eta \Leftarrow \mathcal{C}}{\Delta \vdash \alpha \not\equiv \eta \Rightarrow \neg(\mathcal{C})}$$

Equivalence of existential types: $\boxed{\Delta \vdash \Xi \equiv \Xi}$

$$\frac{\Delta \vdash \Xi \sqsubseteq \Xi' \quad \Delta \vdash \Xi' \sqsubseteq \Xi}{\Delta \vdash \Xi \equiv \Xi'}$$

Free variables in given types: $\boxed{FV(\bar{\alpha}) = \{\bar{P}\}}$

$$FV(\bar{\alpha}) = \bigcup \{FV'(\alpha)\}$$

$$\begin{array}{ll} FV'(P) = \{P\} & FV'(\text{Any}) = \{\} \\ FV'(T[\bar{\alpha}]) = FV(\bar{\alpha}) & FV'(\text{Object}) = \{\} \\ FV'(O[\bar{\alpha}]) = FV(\bar{\alpha}) & FV'(\text{Bottom}) = \{\} \\ FV'((\bar{\alpha})) = FV(\bar{\alpha}) & FV'((\alpha \cup \eta)) = FV(\alpha, \eta) \\ FV'((\alpha \rightarrow \rho)) = FV(\alpha, \rho) & FV'((\alpha \cap \eta)) = FV(\alpha, \eta) \end{array}$$

Figure 15. Miscellaneous Typing Judgments

Existential subtyping:

$$\boxed{\begin{array}{l} \Delta \vdash \Xi \lesssim \Xi \text{ using } \sigma \\ \Delta \vdash \Xi \sqsubseteq \Xi \\ \Delta \vdash \Xi \sqsubseteq \Xi \text{ using } \sigma \end{array}}$$

$$\Delta' = \Delta \cup \{\{\bar{\chi}\} <: \text{invariant } P <: \{\bar{\eta}\}\}$$

$$\{\bar{P}\} \cap FV(\bar{\chi}', \bar{\eta}', \alpha') = \emptyset \quad \sigma = [\gamma/Q]$$

$$\frac{\Delta' \vdash \bar{\alpha}\bar{\chi}' <: \gamma \quad \Delta' \vdash \gamma <: \bar{\alpha}\bar{\eta}' \quad \Delta' \vdash \alpha <: \sigma\alpha'}{\Delta \vdash \exists[\{\bar{\chi}\} <: P <: \{\bar{\eta}\}] \alpha \lesssim \exists[\{\bar{\chi}'\} <: Q <: \{\bar{\eta}'\}] \alpha' \text{ using } \sigma} \text{ [E-SUB]}$$

$$\frac{\Delta \vdash \Xi \xrightarrow{\Xi} \Xi' \text{ using } _ \quad \Delta \vdash \Xi' \lesssim \Xi'' \text{ using } \sigma}{\Delta \vdash \Xi \sqsubseteq \Xi'' \quad \Delta \vdash \Xi \sqsubseteq \Xi'' \text{ using } \sigma}$$

Universal subtyping:

$$\boxed{\begin{array}{l} \Delta \vdash \Upsilon \lesssim \Upsilon \text{ using } \sigma \\ \Delta \vdash \Upsilon \sqsubseteq \Upsilon \\ \Delta \vdash \Upsilon \sqsubseteq \Upsilon \text{ using } \sigma \end{array}}$$

$$\Delta' = \Delta \cup \{\{\bar{\chi}'\} <: \text{invariant } Q <: \{\bar{\eta}'\}\}$$

$$\{\bar{Q}\} \cap FV(\bar{\chi}, \bar{\eta}, \alpha) = \emptyset \quad \sigma = [\gamma/\bar{P}]$$

$$\frac{\Delta' \vdash \bar{\alpha}\bar{\chi} <: \gamma \quad \Delta' \vdash \gamma <: \bar{\alpha}\bar{\eta} \quad \Delta' \vdash \sigma\alpha <: \alpha'}{\Delta \vdash \forall[\{\bar{\chi}\} <: P <: \{\bar{\eta}\}] \alpha \lesssim \forall[\{\bar{\chi}'\} <: Q <: \{\bar{\eta}'\}] \alpha' \text{ using } \sigma} \text{ [U-SUB]}$$

$$\frac{\Delta \vdash \Upsilon \lesssim \Upsilon'' \text{ using } \sigma \quad \Delta \vdash \Upsilon' \xrightarrow{\Xi} \Upsilon'' \text{ using } _}{\Delta \vdash \Upsilon \sqsubseteq \Upsilon' \quad \Delta \vdash \Upsilon \sqsubseteq \Upsilon' \text{ using } \sigma}$$

Existential reduction: $\boxed{\Delta \vdash \Xi \xrightarrow{\Xi} \Xi \text{ using } \sigma}$

$$\frac{\Delta \vdash \alpha \not\equiv \text{Bottom} \Rightarrow \mathcal{C} \quad \text{toConstraint}(\{\bar{K}\}) = \mathcal{C}' \quad \Delta \vdash \text{unify}(\mathcal{C} \wedge \mathcal{C}') = (\sigma, \mathcal{C}'') \quad \text{toBounds}(\mathcal{C}'') = \{\bar{K}'\}}{\Delta \vdash \exists[\bar{K}] \alpha \xrightarrow{\Xi} \exists[\bar{K}'] \sigma(\alpha) \text{ using } \sigma}$$

otherwise

$$\frac{}{\Delta \vdash \exists[\bar{K}] \alpha \xrightarrow{\Xi} \exists[\bar{K}] \alpha \text{ using } []}$$

Universal reduction: $\boxed{\Delta \vdash \Upsilon \xrightarrow{\Xi} \Upsilon \text{ using } \sigma}$

$$\frac{\Delta \vdash \exists[\bar{K}] \alpha \xrightarrow{\Xi} \exists[\bar{K}'] \alpha' \text{ using } \sigma}{\Delta \vdash \forall[\bar{K}](\alpha \rightarrow \omega) \xrightarrow{\Xi} \forall[\bar{K}'](\alpha' \rightarrow \sigma(\omega)) \text{ using } \sigma}$$

Intersection of existential types: $\boxed{\Xi \sqcap \Xi}$

$$\frac{\text{distinct}(\bar{P}, \bar{P}') \quad FV(\bar{\chi}, \bar{\eta}, \alpha) \cap \{\bar{P}'\} = FV(\bar{\chi}', \bar{\eta}', \alpha') \cap \{\bar{P}'\} = \emptyset}{\left(\exists[\{\bar{\chi}\} <: P <: \{\bar{\eta}\}] \alpha \right) \sqcap \left(\exists[\{\bar{\chi}'\} <: P' <: \{\bar{\eta}'\}] \alpha' \right) = \exists[\{\bar{\chi}\} <: P <: \{\bar{\eta}\}, \{\bar{\chi}'\} <: P' <: \{\bar{\eta}'\}] (\alpha \cap \alpha')}$$

Figure 16. Typing Rules for Quantified Types. Note that alpha-renaming of type variables may be necessary to apply some of these rules.

No Duplicates Rule: $\Delta \vdash d$ not duplicate of d

$$\frac{\text{name}(d) \neq \text{name}(d')}{\Delta \vdash d \text{ not duplicate of } d'} \quad [\text{NO-DUP-TRIV}]$$

$$\frac{\text{concrete}(d) \neq \text{concrete}(d')}{\Delta \vdash d \text{ not duplicate of } d'} \quad [\text{NO-DUP-ABS-CONC}]$$

$$\frac{\neg(\Delta \vdash \text{dom}(d) \sqsubseteq \text{dom}(d'))}{\Delta \vdash d \text{ not duplicate of } d'} \quad [\text{NO-DUP-NOT-LESS}]$$

$$\frac{\neg(\Delta \vdash \text{dom}(d') \sqsubseteq \text{dom}(d))}{\Delta \vdash d \text{ not duplicate of } d'} \quad [\text{NO-DUP-NOT-GTR}]$$

Meet Rule: $\Delta \vdash d$ meet d ok

$$\frac{\text{name}(d) \neq \text{name}(d')}{\Delta \vdash d \text{ meet } d' \text{ ok}} \quad [\text{MEET-TRIV}]$$

$$\frac{\Delta \vdash (\text{dom}(d) \cap \text{dom}(d')) \sqsubseteq \exists \llbracket \text{Bottom} \rrbracket}{\Delta \vdash d \text{ meet } d' \text{ ok}} \quad [\text{MEET-EXCL}]$$

$$\frac{\Delta \vdash \text{dom}(d) \sqsubseteq \text{dom}(d')}{\Delta \vdash d \text{ meet } d' \text{ ok}} \quad [\text{MEET-SPEC}]$$

$$\frac{\begin{array}{c} d'' \in \Delta \\ \text{name}(d) = \text{name}(d') = \text{name}(d'') \\ \Delta \vdash \text{dom}(d'') \equiv (\text{dom}(d) \cap \text{dom}(d')) \end{array}}{\Delta \vdash d \text{ meet } d' \text{ ok}} \quad [\text{MEET-THIRD}]$$

Return Type Rule: $\Delta \vdash d$ return type wrt d ok

$$\frac{\text{name}(d) \neq \text{name}(d')}{\Delta \vdash d \text{ return type wrt } d' \text{ ok}} \quad [\text{RETURN-TRIV}]$$

$$\frac{\neg(\Delta \vdash \text{dom}(d) \sqsubseteq \text{dom}(d'))}{\Delta \vdash d \text{ return type wrt } d' \text{ ok}} \quad [\text{RETURN-MOOT}]$$

$$\frac{\begin{array}{c} \text{arrow}(d) = \forall \llbracket \bar{\kappa} \rrbracket ((\bar{\alpha}) \rightarrow \rho) \quad \overline{\kappa} = \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \\ \text{arrow}(d') = \forall \llbracket \bar{\kappa}' \rrbracket ((\bar{\alpha}') \rightarrow \rho') \quad \overline{\kappa'} = \{\bar{\chi}'\} <: Q <: \{\bar{\eta}'\} \\ \text{distinct}(\bar{P}, \bar{Q}) \quad \Delta \vdash \text{dom}(d) \sqsubseteq \text{dom}(d') \text{ using } \sigma \\ \Delta \vdash \forall \llbracket \bar{\kappa} \rrbracket ((\bar{\alpha}) \rightarrow \rho) \sqsubseteq \forall \llbracket \bar{\kappa} \rrbracket \sigma(((\bar{\alpha}) \cap (\bar{\alpha}')) \rightarrow \rho') \end{array}}{\Delta \vdash d \text{ return type wrt } d' \text{ ok}} \quad [\text{RETURN-TEST}]$$

Ancestors Rule: $\Delta; \bar{\chi} \vdash T[\bar{\alpha}]$ and $T[\bar{\alpha}]$ ancestors ok

$$\frac{T \neq T'}{\Delta; \bar{\chi} \vdash T[\bar{\alpha}] \text{ and } T'[\bar{\eta}] \text{ ancestors ok}} \quad [\text{ANC-DIFF-TRAIT}]$$

$$\frac{\begin{array}{c} \text{trait } T[\bar{V} P <: \{\bar{\gamma}\}] \text{ -- end } \in \Delta \quad T[\bar{\gamma}] \in \{\bar{\chi}\} \\ \Delta \vdash T[\bar{\gamma}] <: T[\bar{\alpha}] \quad \Delta \vdash T[\bar{\gamma}] <: T[\bar{\eta}] \end{array}}{\Delta; \bar{\chi} \vdash T[\bar{\alpha}] \text{ and } T[\bar{\eta}] \text{ ancestors ok}} \quad [\text{ANC-SAME-TRAIT}]$$

Figure 17. The No Duplicates Rule, the Meet Rule, the Return Type Rule, and the Ancestors Rule. Alpha-renaming of type variables may be necessary to apply these rules.

Figures 13 and 14. All other rules are obtained as follows: For each rule in the figures that is not in a section marked “bound variables,” make a new rule by replacing each occurrence of a relation symbol with its negation, replacing $\wedge$ with $\vee$ and vice versa. For example, from the rule:

$$\frac{\begin{array}{c} \Delta \vdash \alpha <: \eta \Leftarrow \mathcal{C} \\ \Delta \vdash \alpha <: \chi \Leftarrow \mathcal{C}' \end{array}}{\Delta \vdash \alpha <: (\eta \cap \chi) \Leftarrow \mathcal{C} \wedge \mathcal{C}'}$$

we derive:

$$\frac{\begin{array}{c} \Delta \vdash \alpha \not<: \eta \Leftarrow \mathcal{C} \\ \Delta \vdash \alpha \not<: \chi \Leftarrow \mathcal{C}' \end{array}}{\Delta \vdash \alpha \not<: (\eta \cap \chi) \Leftarrow \mathcal{C} \vee \mathcal{C}'}$$

The negative judgment should not be confused with the derived contrapositive judgment $\Delta \vdash T \not\equiv T' \Rightarrow \mathcal{C}$ given in Figure 15, for the two judgments handle bound type variables very differently. Intuitively, the negative assertion $\Delta \vdash T \not\equiv T' \Leftarrow \mathcal{C}$ computes the constraint $\mathcal{C}$ that satisfies the inequivalence for an arbitrary instantiation of the type variables bound in Δ , whereas the contrapositive assertion $\Delta \vdash T \not\equiv T' \Rightarrow \mathcal{C}$ computes the constraint $\mathcal{C}$ that holds for any instantiation of Δ such that the inequivalence is true.

Figure 15 gives rules that define type equivalence in terms of subtyping and define unconditional versions of the subtyping, type exclusion, and type equivalence relationships.

Figure 16 gives rules for deciding whether one existentially quantified type is a subtype of another, and whether one universally quantified type is a subtype of another. This figure introduces new variants of these subtyping judgments that make available the substitution σ deduced to justify the judgment. This allows us to make a substantial practical improvement to the Return Type Rule in Figure 17.

Here is the old meet rule from [3]:

$$\frac{\begin{array}{c} \text{arrow}(d) = \forall \llbracket \bar{\kappa} \rrbracket ((\bar{\alpha}) \rightarrow \rho) \quad \overline{\kappa} = \{\bar{\chi}\} <: P <: \{\bar{\eta}\} \\ \text{arrow}(d') = \forall \llbracket \bar{\kappa}' \rrbracket ((\bar{\alpha}') \rightarrow \rho') \quad \overline{\kappa'} = \{\bar{\chi}'\} <: Q <: \{\bar{\eta}'\} \\ \text{distinct}(\bar{P}, \bar{Q}) \quad \Delta \vdash \text{dom}(d) \sqsubseteq \text{dom}(d') \end{array}}{\Delta \vdash \forall \llbracket \bar{\kappa} \rrbracket ((\bar{\alpha}) \rightarrow \rho) \sqsubseteq \forall \llbracket \bar{\kappa} \rrbracket \sigma(((\bar{\alpha}) \cap (\bar{\alpha}')) \rightarrow \rho')} \quad [\text{OLD-RETURN-TEST}]$$

The problem here is that the last premise requires a difficult inference, because the two universal types have different sets of type parameters. The version presented here as rule [RETURN-TEST] in Figure 17 takes advantage of the fact that the preceding premise $\Delta \vdash \text{dom}(d) \sqsubseteq \text{dom}(d')$ has already had to compute a substitution that resolves the differences between the type parameters of the two functions or methods in question; replacing $\Delta \vdash \text{dom}(d) \sqsubseteq \text{dom}(d')$ with $\Delta \vdash \text{dom}(d) \sqsubseteq \text{dom}(d')$ using σ makes this substitution σ available for use in the final premise, which is then much easier to resolve.

7. Run-time Type Inference

We now draw attention to a detail of rule [E-SUB] in Figure 16: in order to carry out this inference rule, it is necessary to guess, compute, or pluck from thin air a substitution σ that validates the judgment in a type context in which type parameters $\{\bar{P}\}$ have been added. This process is not a great burden when performed statically (“at compile time”), but can be a significant source of inefficiency at run time. We now make a series of observations.

The function *msav* can be implemented in more than one way. Rule [MSAV-STANDARD] in Figure 10 defines it in a straightforward manner: the most specific of the applicable procedures that are visible. Rule [MSAV-ALTERNATE] at the bottom of the same figure takes an alternate approach: the determination of whether one procedure is more specific than another can be done statically, and the No Duplicates Rule ensures that there are no cycles in the relationship (therefore “more specific” is a partial order). So the set of methods visible at any call site can be statically sorted, using a topological sort, into an ordered list such that if procedure 1 is more specific than procedure 2 then procedure 1 appears earlier in the list than procedure 2. Then all *msav* needs to do at run time is to find the first method in this sorted list that is applicable. Therefore the only use of rule [E-SUB] that occurs at run time is the one in rule [APPLICABLE].

Regarding the use of rule [E-SUB] in rule [APPLICABLE], some further observations: the existential type to the left of the “ $\sqsubseteq$ ” has no type parameters. This is related to the fact that when *msav* is used at run time, the tuple $(\bar{x})$ is in fact a tuple of ilks, which contain no type parameters. An even more subtle point is this proposition:

Proposition 8 When *applicable* is called at “run time” (from rule [R-FUNCTION] or [R-METHOD]), the type of each value parameter x of each function or method considered is of the form $(\bar{s})$; that is, it is a *signature type*.

This fact implies that when matching argument value ilks against procedure parameter types, type parameters P will not occur in certain difficult situations, namely within operands of unions and intersections.

The task is, nevertheless, to compute the substitution σ (an appropriate assignment of types to the type parameters of the function or method). In the following sections we present algorithms that exploit the observations of this section to achieve that task.

8. Dispatch Implementation Strategy

We use the term *entrypoint* to refer to any of the concrete definitions of a function or method of a given name that are statically visible at a call site under consideration. The task of dynamic dispatch, given a set of argument values at run time, is to select an appropriate entrypoint.

The dispatch implementation strategy outlined here is applicable to any language for which preferences among the

entrypoints of an overloaded function are described by a statically determinable partial order. For Fortress, the partial order is determined purely by specificity (the subtype relationship on the domains of the procedures), but other languages might rely instead, or additionally, on user-specified preferences or on the order in which traits or methods are declared or mentioned in the program text.

By testing for applicability in most-to-least specific order, dynamic dispatch consists simply of invoking the dispatch semipredicate for each entrypoint in the list until some semipredicate succeeds. The goal is then to keep dispatch semipredicates cheap to execute, and in particular to keep simple, common cases especially cheap. We note that if a program is well-formed—that is, passes static type checking—then it is not possible for all of the semipredicates to fail. Therefore it is never necessary to call the dispatch semipredicate for the last entrypoint in a list—and if there is only one entrypoint in the list (an important common case), then no dispatch semipredicate need be called at all.

8.1 Dispatch semipredicate

The dispatch semipredicate for an entrypoint takes an ilk (a tuple of the ilks of the argument values) as input, and either fails (returns false) if the entrypoint is not applicable, or returns a non-false result if the entrypoint is applicable, and that non-false result is in fact a set of type bindings for the static type parameters of that entrypoint. (If the entrypoint has no static type parameters, then the result is an empty set of type bindings, which is not the same as “false”).

We actually describe the dispatch semipredicate computation as a function of *two* arguments. The first argument is the ilk already mentioned; the second is a signature type (a tuple of the declared types of the value parameters of the entrypoint). Note that if methods rather than functions are under consideration, then the signature type may involve union and intersection types because of substitution of such types for type parameters of the trait or object in which the method was declared; however, type parameters of the method cannot appear within the operands of such unions and intersections. To put it another way, all union types and intersection types appearing within a signature type are necessarily ground types. This fact simplifies the dispatch process. The ilk is not known until run time, but the signature type is known at compile time. We also note that, for each occurrence of a type parameter within a signature type, the variance of the occurrence may be determined statically. Therefore the dispatch process as we describe it here may be understood as a staged computation, and portions that rely only on the signature type may be “unwound” or “inlined” at compile time.

8.2 Normal forms for types

We assume that both signature types and ground types are kept in a canonical form, namely sorted disjunctive normal form. For this purpose we regard $\cup$ and $\cap$ as n-ary operators

rather than binary operators. The production of such a normal form from a given type may be described in two steps:

1. (*Flatten constructed types with certain union/intersection arguments.*) At type parameter positions where G is covariant, replace $G[\dots, \cap(\bar{\alpha}), \dots]$ with $\cap(G[\dots, \alpha, \dots])$, and at type parameter positions where G is contravariant, replace $G[\dots, \cup(\bar{\alpha}), \dots]$ with $\cap(G[\dots, \alpha, \dots])$. (See Appendix B for a proof sketch that in each case a type is replaced with an equivalent type. The proof relies on the Ancestor Meet Rule.)
2. (*Sorted DNF.*) Reduce the entire type, as well as any subexpression involving union or intersection types, to sorted disjunctive normal form, wherein the operands of every $\cap$ operator and then every $\cup$ operator are sorted according to some fixed total order (such as lexicographic). Before the sorting, unions and intersections are simplified by using absorption laws, as well as the rule that any intersection containing two types that exclude each other may be replaced with Bottom.

Because types are expressed in canonical form, one can know, for example, that if s and s' are operands of the same $\cap$ or $\cup$ operator, then neither is a subtype of the other.

8.3 Simplified version: no contravariance or $\cap$

The simplest version of this algorithm supports covariant and invariant type parameters, but not contravariant type parameters or domain-contravariant arrow types. (Domain-invariant arrow types can be handled by this simplified version). Union types are handled, but intersection types are not. In the absence of contravariance, the semipredicate is organized into two phases:

1. Match ilk against signature type. This proceeds by structural recursion on the signature type, keeping track of variance as we go. (Recall that because variance depends only on the signature type and not the ilk, it can be computed statically.)
2. Propagate lower limits through type constraints, ordered from most-constrained to least-constrained. Because of the way that type parameters of functions and methods are scoped in Welterweight Fortress, this means that type parameters will be processed in right-to-left order. The goal of overloaded function dispatch is to discover the most specific applicable entrypoint, so inference also aims to obtain the most specific instantiation of an applicable entrypoint.

8.3.1 Match ilk against signature type

The recursive MATCH procedure is presented here as if it were called as a subroutine of the dispatch semipredicate. However, most of the code is conditional on the structure of the signature type of the entrypoint and can be rendered in an open-coded form by a compiler, including only those cases required by a particular entrypoint's signature.

MATCH takes three parameters: a ground type g , the current variance V (encoded as +1 for covariant, 0 for invariant, and -1 for contravariant), and a signature type s . If the current variance V is +1, then we require $g <: s$ for success; if $V = 0$, then we require $g \equiv s$; if $V = -1$, then we require $s <: g$.

Before the MATCH procedure is called, the dispatch semipredicate establishes, for each type parameter P of the entrypoint, lower and upper bound constraint sets $\mathcal{L}_P$ and $\mathcal{U}_P$. For a function entrypoint, $\mathcal{L}_P$ is initially empty; for a method entrypoint, $\mathcal{L}_P$ initially contains all the declared lower bounds of the type parameter (which are all ground types at this point, because trait parameters have been substituted for and method parameters cannot appear in lower bounds). For both kinds of entrypoint, $\mathcal{U}_P$ initially contains all the declared upper bounds of the type parameter that happen to be ground types. (Upper bounds that are parametric signature types are processed in a later step.)

If the type and signature can be properly related, then MATCH succeeds after adding further necessary constraints to the lower and upper bound sets. If the type and signature cannot be properly related, then MATCH fails, indicating that the entrypoint is not applicable.

For example, the dispatch predicate for this entrypoint:

$$f[P, Q, S](a: G[P], b: H[P, Q], c: J[Q, S]): _ = _$$

allocates six empty constraint sets $\mathcal{L}_P, \mathcal{L}_Y, \mathcal{L}_S, \mathcal{U}_P, \mathcal{U}_Y, \mathcal{U}_S$, then invokes $\text{MATCH}(k, +1, (G[P], H[P, Q], J[Q, S]))$ where k is the argument ilk. (Note: all ilks are ground types.)

- 1: **function** MATCH(g :Type, V :Variance, s :Type)
- 2: **if** s is a ground type **then**
- 3: **if** $V \geq 0$ **then** $\otimes$ co/invariant, require $g <: s$
- 4: **if** $\neg(g <: s)$ **then** dispatch fails
- 5: **if** $V \leq 0$ **then** $\otimes$ contra/invariant, require $s <: g$
- 6: **if** $\neg(s <: g)$ **then** dispatch fails
- 7: **else if** s is a type parameter P **then**
- 8: **if** $V \geq 0$ **then** $\otimes$ co/invariant, require $g <: P$
- 9: insert g into $\mathcal{L}_P$
- 10: **if** $V \leq 0$ **then** $\otimes$ contra/invariant, require $P <: g$
- 11: insert g into $\mathcal{U}_P$
- 12: **else if** s is an arrow type $s_{\text{domain}} \rightarrow s_{\text{range}}$ **then**
- 13: **if** g is an arrow type $g_{\text{domain}} \rightarrow g_{\text{range}}$ **then**
- 14: MATCH($g_{\text{domain}}, 0, s_{\text{domain}}$)
- 15: MATCH($g_{\text{range}}, V, s_{\text{range}}$)
- 16: **else** dispatch fails
- 17: **else if** s is a tuple type $(s_1, \dots, s_m)$ **then**
- 18: **if** g is a same-length tuple type $(g_1, \dots, g_m)$ **then**
- 19: **for** $1 \leq j \leq m$ **do**
- 20: MATCH(g_j, V, s_j)
- 21: **else** dispatch fails
- 22: **else if** g is a union of types $g_1, \dots, g_m$ **then**
- 23: $\otimes$ s is a constructed signature type
- 24: **if** $V = 0$ **then** dispatch fails

```

25:   else
26:     for  $1 \leq j \leq m$  do
27:       MATCH( $g_j, V, s$ )
28:   else
29:      $\otimes$   $s$  is a constructed signature type  $T[s'_1, \dots, s'_m]$ 
30:      $\otimes$   $g$  is not a union or intersection
31:     if  $g$  is a constructed type that has some ancestor
32:       of the form  $T[\square]$  then
33:       let  $g' =$  the minimal ancestor  $T[g_1, \dots, g_m]$  of  $g$ 
34:       that has the form  $T[\square]$ 
35:       if  $(V = 0) \wedge (g \neq g')$  then dispatch fails
36:       let  $(V_1, \dots, V_m) =$  variances of  $T$ 's parameters
37:       for  $1 \leq j \leq m$  do
38:         MATCH( $g_j, V \times V_j, s'_j$ )
39:     else dispatch fails

```

The loop on line 26 deals with the case where a union type g must be a subtype of a constructed type $G[\square]$. The loop ensures that each term g_j in the union contributes to the constraints on the type parameters in s .

8.3.2 Propagate lower limits through type constraints

The constraint propagation and checking algorithm uses a type operation “LSUB” (Least Single Upper Bound). For a lattice, this is lattice join, implemented in the Fortress type system as a union type (which may then be simplified). In programming languages with single inheritance, the LSUB of two constructed types can be defined as their unique least common ancestor; in other programming languages that support type multiple inheritance LSUB(T, U) may be defined in some other way, for instance as some least declared supertype dominating T and U , perhaps with a tie-breaking rule. The LSUB operation must obey the property that for a constructed type G with a covariant parameter position, $\text{LSUB}\{G[\dots, \alpha, \dots]\} <: G[\dots, \text{LSUB}\{\bar{\alpha}\}, \dots]$.

The constraint propagation algorithm merely ensures that, whatever the definition of LSUB is, if a consistent set of type assignments can be discovered, the most specific one will be found. Suppose that the entrapoint of interest has n type parameters; we process them from right to left.

```

1: for  $j$  sequentially from  $n$  down to 1 do
2:    $l_j \leftarrow \text{LSUB}(\mathcal{L}_{P_j})$ 
3:   if  $l_j \not\prec$ : Bottom then
4:     for each declared upper bound  $\xi$  of  $P_j$  do
5:       if  $\xi$  is a parametric signature type then
6:         MATCH( $l_j, +1, \xi$ )
7:     for each  $u$  in  $\mathcal{U}_{P_j}$  do
8:       if  $l_j \not\prec$ :  $u$  then
9:         dispatch fails

```

Each iteration of the outer loop begins by establishing the best possible lower bound l_j for P_j . Because the parameters are processed right-to-left, P_j will not appear in subsequent constraints and thus its lower bound cannot be further modified. Line 6 invokes MATCH to ensure that the lower bound

of P_j is structurally able to be a subtype of each declared upper bound ξ , and to propagate necessary bounds to other type parameters that have not yet been checked and inferred.

If the match and propagation processes complete successfully, then l_j is the type inferred for P_j , and the dispatch semipredicate returns the substitution $[l/P]$.

(It should be noted that the Return Type Rule may require one final adjustment step, which may replace each l_j in the computed substitution with a type that is a supertype of l_j but still a subtype of every type in $\mathcal{U}_j$. However, this step always succeeds and can be performed after the entrapoint is been chosen by the dispatch process, so we do not consider it to be a part of the dispatch semipredicate.)

8.4 Run-time type representations and operations

The run-time metadata for trait and object types must support the following three operations, ideally with low cost:

- Is α a subtype of η ?
- Find minimal instance of G that is an ancestor of k .
- Meet and join (intersection and union) operations.

Here we describe an implementation strategy.

For each type α , the “longest erased path to Any,” $\text{LEPA}(\alpha)$, is calculated. $\text{LEPA}(\text{Any})$ is defined to be zero; if α is Object or an arrow or tuple type, $\text{LEPA}(\alpha)$ is defined to be 1. If α is a constructed type C , $\text{LEPA}(\alpha)$ is defined to be 1 plus the maximum of 1 and all values $\text{LEPA}(U)$ such that some instance of U appears in the extends clause of the declaration of the trait or object of which C is an instance.

For constructed type C , $\text{ARGS}(C)$ is the sequence of type arguments in C , and $\text{VARIANCES}(C)$ is the corresponding sequence of variances (the same sequence for all instances of the same trait or object).

For a given trait or object type C , all its ancestors are encoded into an array $\text{SUPERS}(C)$ of maps from type names to types. Entry $\text{SUPERS}(C)_i$ contains the map of supertypes of C whose LEPA is i ; the domain of the map is the names of the ancestors, and each name is mapped to the (unerased) minimal instance of that generic type among the ancestors of C . The SUPERS array for C includes an entry for C itself: $\text{SUPERS}(C)_{\text{LEPA}(C)}(\text{name}(C)) = C$. Each map may of course be implemented as a simple list of pairs, a sorted list, an ordered binary tree, a hashmap, or other suitable data structure. Given this data structure, the minimal instance of a type constructor G among the ancestors of C is $\text{SUPERS}(C)_{\text{LEPA}(G)}(G)$.

A subtype query is slightly more involved. (For lack of space, we elide the handling of tuples, arrows, Any, Object, Bottom, and unions and intersections and show just the code for comparing two constructed types.)

```

1: function SUBTYPE( $\alpha, \eta$ )
2:    $\otimes$  Return true if  $\alpha$  is a subtype of  $\eta$ 
3:   if ... then
4:     (handling of tuples, arrows, etc., elided here)

```

```

5:  else if  $\text{LEPA}(\alpha) < \text{LEPA}(\eta)$  then false
6:  else if  $\text{name}(\eta) \notin \text{SUPERS}(\alpha)_{\text{LEPA}(\eta)}$  then false
7:  else
8:     $U = \text{ARGS}(\eta)$ 
9:     $\text{candidate} = \text{SUPERS}(\alpha)_{\text{LEPA}(\eta)}(\text{name}(\eta))$ 
10:    $Y = \text{ARGS}(\text{candidate})$ 
11:    $\text{vars} = \text{VARIANCES}(\text{candidate})$ 
12:   var  $\text{result} : \text{Boolean} = \text{true}$ 
13:   for  $i$  sequentially from 1 up to  $|Y|$  do
14:     case  $\text{vars}_i$  of
15:        $+1 \Rightarrow \text{result} \leftarrow \text{result} \wedge \text{SUBTYPE}(Y_i, U_i)$ 
16:        $0 \Rightarrow \text{result} \leftarrow \text{result} \wedge (Y_i = U_i)$ 
17:        $-1 \Rightarrow \text{result} \leftarrow \text{result} \wedge \text{SUBTYPE}(U_i, Y_i)$ 
18:    $\text{result}$ 

```

9. Variations

The algorithm just described works with covariant and invariant generic types, union types, and a set of constraints allowed on generic type parameters. We can relax some of these restrictions and accept more varied inputs.

9.1 Intersection types from typecase

Intersection types can arise in the Welterweight Fortress match construct, and such types can then make their way into ilks such as $\text{List}[\text{Serializable} \cap \text{Cloneable}]$.

In the MATCH algorithm (no pun intended; “match” and “MATCH” are in different fonts), intersection types can introduce complications when a constructed signature type, for example $G[P]$, must covariantly match an intersection type $(g \cap g')$; this requires $(g \cap g') <: G[P]$ for success. This will be true if either $g <: G[T]$ or $g' <: G[T]$ (or both). In principle, constraint checking must be performed for each of these combinations, with the most specific (both) case being preferred. If neither clause matches, then dispatch fails; if g matches but not g' (or vice versa), then there are no alternatives and no combinatorial explosion. But if both clauses match locally, it may be that only one of the two is compatible with other constraints inferred for the entrypoint under consideration, so on the face of it there is a need to check all possible combinations separately.

However, the restrictions on how traits may be extended prohibit mutually exclusive constraints from the two clauses of the intersection. For the match construct, the intersection type results from an actual value with ilk k that must satisfy all structural constraints on types. Suppose that g and g' both extend instantiations of G , so $k <: g <: G[g_1]$ and $k <: g' <: G[g_2]$. If G is invariant, then g_1 and g_2 must be equal, and therefore the constraints for the two clauses are compatible. If G is covariant, then g_1 and g_2 may not be the same. Because $g <: G[g_1]$ and $g' <: G[g_2]$, we know that $g \cap g' <: G[g_1] \cap G[g_2] = G[g_1 \cap g_2]$. This allows us to conclude that $g_1 \cap g_2$ is a valid lower bound for g . Because the intersection is more specific than both g_1 and g_2 , it also

allows all combinations of g_1 and g_2 without the overhead of testing them separately.

Processing the clauses of an intersection type extending a generic signature does require one modification to the MATCH algorithm; if a clause fails to match, it does not by itself indicate dispatch failure for the current entrypoint. Furthermore, any partial set of lower and upper bound constraints accumulated during the attempted match of a clause must be discarded. For a successful match, the lower bound sets computed for each of the clauses must all be intersected; effectively, the resulting bound sets are each the intersection of unions, which must then be combined with the existing lower bound sets. Upper bound sets will be merged to form the union of intersections. All this is handled by inserting lines 2–29 of the following code just before line 28 of the MATCH algorithm shown in Section 8.3.1.

```

1:   $\langle \text{intersection subtype of constructed signature type?} \rangle =$ 
2:  else if  $g$  is an intersection of types  $g_1, \dots, g_m$  then
3:     $\otimes s$  is a constructed signature type
4:     $\otimes n$  is the number of method type parameters
5:    if  $V = 0$  then dispatch fails
6:    else
7:      for  $1 \leq i \leq n$  do
8:        Save existing bound sets  $\mathcal{L}_{P_i}$  and  $\mathcal{U}_{P_i}$ 
9:         $\text{LBT}_{P_i} \leftarrow \text{Any}$   $\otimes$  Lower bound type
10:        $\text{UBT}_{P_i} \leftarrow \text{Bottom}$   $\otimes$  Upper bound type
11:       var  $\text{anymatch} = \text{false}$ 
12:       for  $1 \leq j \leq m$  do
13:         for  $1 \leq i \leq n$  do
14:            $\mathcal{L}_{P_i} \leftarrow \{ \}$ 
15:            $\mathcal{U}_{P_i} \leftarrow \{ \}$ 
16:         try
17:            $\text{MATCH}(g_j, V, s)$   $\otimes$  sole use of  $j$  is here
18:           for  $1 \leq i \leq n$  do
19:              $\text{LBT}_{P_i} \leftarrow \text{LBT}_{P_i} \cap \bigcup_{g' \in \mathcal{L}_{P_i}} g'$ 
20:              $\text{UBT}_{P_i} \leftarrow \text{UBT}_{P_i} \cup \bigcap_{g' \in \mathcal{U}_{P_i}} g'$ 
21:            $\text{anymatch} \leftarrow \text{true}$ 
22:         catch dispatch failure
23:       if  $\neg \text{anymatch}$  then dispatch fails
24:       for  $1 \leq i \leq n$  do
25:         Restore saved bound sets  $\mathcal{L}_{P_i}$  and  $\mathcal{U}_{P_i}$ 
26:         if  $\text{Any} \not<: \text{LBT}_{P_i}$  then
27:            $\mathcal{L}_{P_i} \leftarrow \mathcal{L}_{P_i} \cup \{ \text{LBT}_{P_i} \}$ 
28:         if  $\text{UBT}_{P_i} \not<: \text{Bottom}$  then
29:            $\mathcal{U}_{P_i} \leftarrow \mathcal{U}_{P_i} \cup \{ \text{UBT}_{P_i} \}$ 

```

9.2 Contravariance

Adding contravariance to the type system creates the possibility of a combinatorial blowup in constraint matching when a union type must match a generic signature in a contravariant context. Because of space constraints, we do not

present a complete solution here, but we make some general remarks about the nature of the difficulty.

Consider the problem of matching an ilk $(g \cup g')$ with the signature type $G[\bar{s}]$; that is, we are trying to determine whether $G[\bar{s}] <: (g \cup g')$, where g and g' are ground constructed types. Then the two subproblems are $G[\bar{s}] <: g$ and $G[\bar{s}] <: g'$. There are a few easy cases. If both subproblems fail to match, the overall match fails. If just one subproblem fails to match, then the result is exactly the successful match of the other subproblem. If either subproblem succeeds with an empty set of constraints, then the result is the empty set of constraints.

This leaves the case where both subproblems match, and both subproblems produce non-trivial constraints. This results when $g = H[\dots]$ and $g' = J[\dots]$ and $G[\bar{\gamma}]$ is declared to (transitively) extend $H[\bar{\alpha}]$ and $J[\bar{\eta}]$, where some elements of $\bar{\gamma}$ appear in $\bar{\alpha}$ and in $\bar{\eta}$, and for each element of $\bar{\gamma}$ appearing in $\bar{\alpha}$ and/or $\bar{\eta}$, the variance for that element is the same in $\bar{\gamma}$, $\bar{\alpha}$, and $\bar{\eta}$. This is required by the rules for variance context: covariant type parameters may appear only in covariant contexts, contravariant type parameters may appear only in contravariant contexts. Note that it is also possible that H and J are the same name, and either or both may be the same name as G .

Suppose H and J are both G . Then the two subproblems are $G[\bar{P}_1] <: G[\bar{U}]$ and $G[\bar{P}_2] <: G[\bar{V}]$. Because $G[\bar{U}] \cup G[\bar{V}]$ is assumed to be in canonical form, there is no $\bar{P}$ such that $G[\bar{P}] = G[\bar{U}] \cup G[\bar{V}]$. This is a hard subcase; it is necessary to explore each possibility separately, and if such choices are cascaded, the result can be an exponential blowup.

If we have only that H and J are the same name, different from G , this still leads to a hard subcase by similar (but more detailed) reasoning.

On the other hand, suppose neither H nor J is G . In that case, $G[\bar{P}_1] <: H[\bar{U}]$ and $G[\bar{P}_2] <: J[\bar{V}]$. Because H and J are not G , canonical form of the union does not preclude the possibility that $G[\bar{P}_1]$ and $G[\bar{P}_2]$ are related. When that occurs (for example, if $G[\bar{P}_1] <: G[\bar{P}_2]$), then one subproblem subsumes the other, and the solution for the larger problem is the set of constraints from the subsuming subproblem.

9.3 No union types

It is possible to use a type system in which union types do not appear. Many programming languages pseudo-join two types by choosing their nearest dominator in the type hierarchy instead of forming a union type. This allows contravariant generic types without the combinatorial cost of a contravariant generic-to-union match.

10. Related Work

Castagna *et al.* proposed rules for defining overloaded functions to ensure type safety [5]. The type hierarchy is required

to be a meet semilattice (to ensure that any two types have a greatest lower bound).

Millstein and Chambers devised the language *Dubious* to study how to modularly ensure safety for overloaded functions with symmetric multiple dynamic dispatch (*multimethods*) in a type system supporting multiple inheritance [11, 12]. With Clifton and Leavens, they developed Multi-Java [6], an extension of Java with Dubious' semantics for multimethods. Lee and Chambers presented F(EML) [9], a language with classes, symmetric multiple dispatch, and parameterized modules. In previous work, we built on the work of Millstein and Chambers to give modular rules for a core of the Fortress language [1]; for detailed comparison of modularity and dispatch for these systems, see the related work section of our previous paper [2].

None of the systems described above support polymorphic functions or types. F(EML)'s parameterized modules (*functors*) provide a form of parametricity but they cannot be applied implicitly.

Overloading and multiple dispatch in the context of polymorphism has previously been studied by Bourdoncle and Merz [4]. Their system, $ML_{\leq}$, integrates parametric polymorphism, class-based object orientation, and multimethods, but lacks multiple inheritance. Each multimethod (overloaded set) requires a unique specification (principal type), which greatly simplifies the checking of their equivalent of the Return Type Rule: the return type of each definition needs to be compared only with the return type of the principal type. Also, their type system does not provide any exclusion relation. On the other hand, $ML_{\leq}$, unlike Fortress allows variance annotations on type constructors.

Litvinov [10] developed a type system for the Cecil language, which supports bounded parametric polymorphism and multimethods. Because Cecil has a type-erasure semantics, statically checked parametric polymorphism has no effect on run-time dispatch.

The relatively new language ParaSail [15] also constrains the scope of type parameters, prohibiting a bound on a type parameter from referring to type parameter bindings to its left, so that dynamic type inference can be fast. However, ParaSail apparently does not support covariant and contravariant types parameters.

References

- [1] Eric Allen, David Chase, Joe Hallett, Victor Luchangco, Jan-Willem Maessen, Sukyoung Ryu, Guy L. Steele Jr., and Sam Tobin-Hochstadt. The Fortress Language Specification Version 1.0, March 2008.
- [2] Eric Allen, J. J. Hallett, Victor Luchangco, Sukyoung Ryu, and Guy L. Steele Jr. Modular multiple dispatch with multiple inheritance. In *SAC '07*, 2007.
- [3] Eric Allen, Justin Hilburn, Scott Kilpatrick, Victor Luchangco, Sukyoung Ryu, David Chase, and Guy Steele. Type checking modular multiple dispatch with parametric polymorphism and multiple inheritance. In *Proceedings*

of the 2011 ACM International Conference on Object Oriented Programming, Systems, Languages, and Applications, OOPSLA '11, pages 973–992, New York, NY, USA, 2011. ACM.

- [4] François Bourdoncle and Stephan Merz. Type checking higher-order polymorphic multi-methods. In *POPL '97*, 1997.
- [5] Giuseppe Castagna, Giorgio Ghelli, and Giuseppe Longo. A calculus for overloaded functions with subtyping. *Information and Computation*, 117(1):115–135, 1995.
- [6] Curtis Clifton, Todd Millstein, Gary T. Leavens, and Craig Chambers. MultiJava: Design rationale, compiler implementation, and applications. *ACM TOPLAS*, 28(3), 2006.
- [7] James Gosling, Bill Joy, Guy Steele, and Gilad Bracha. *The Java Language Specification, Third Edition*. Addison-Wesley Longman, Amsterdam, 3 edition, June 2005.
- [8] Jieung Kim and Sukyoung Ryu. Coq mechanization of Featherweight Fortress with multiple dispatch and multiple inheritance. In *Proceedings of the First International Conference on Certified Programs and Proofs, CPP'11*, pages 264–279, Berlin, Heidelberg, 2011. Springer-Verlag.
- [9] Keunwoo Lee and Craig Chambers. Parameterized modules for classes and extensible functions. In *ECOOP '06*, 2006.
- [10] Vassily Litvinov. Constraint-based polymorphism in Cecil: Towards a practical and static type system. In *OOPSLA '98*, 1998.
- [11] Todd Millstein and Craig Chambers. Modular statically typed multimethods. *Information and Computation*, 175(1):76–118, 2002.
- [12] Todd David Millstein. *Reconciling software extensibility with modular program reasoning*. PhD thesis, University of Washington, 2003.
- [13] Martin Odersky. *The Scala Language Specification, Version 2.7*. EPFL Lausanne, Switzerland, 2009.
- [14] Daniel Smith and Robert Cartwright. Java type inference is broken: can we fix it? In *OOPSLA '08*, 2008.
- [15] S. Tucker Taft. Multicore programming in ParaSail: Parallel specification and implementation language. In *Proceedings of the 16th Ada-Europe International Conference on Reliable Software Technologies, Ada-Europe'11*, pages 196–200, Berlin, Heidelberg, 2011. Springer-Verlag.

A. Detailed Description of Notation

We use the term *enclosers* to refer to symbols that are always used in properly nested matched pairs; examples are parentheses $()$, brackets $[]$ and $\llbracket \rrbracket$, and braces $\{\}$. In this paper we also treat the keyword tokens *trait* and *end* as matching enclosers, and likewise *object* and *end*. The tokens and symbols between a matched pair of enclosers are said to be *enclosed* by the pair.

We use the term *monogram* to refer to a single letter (Latin or Greek) that, rather than being used for decorative purposes, is itself possibly “decorated” with one or more prime marks and/or a sequence of one or more subscripts. Examples of monograms are x , β , e' , α_2 , and τ'_{1527} .

We write $\bar{x}$ as shorthand for a possibly empty comma-separated sequence $x_1, x_2, \dots, x_n$ for some freely chosen nonnegative integer n ; thus $\bar{x}$ may expand to “” or “ x_1 ” or “ x_1, x_2 ” or “ x_1, x_2, x_3 ” or “ x_1, x_2, x_3, x_4 ” and so on. More generally, for any expression, that same expression with an overbar is shorthand for a possibly empty comma-separated sequence of copies of that expression (indexed by i) with three transformations applied to each copy, in order:

1. First, any subexpression that is not underlined and has an arrow $\xrightarrow{\circ}$ above it is replaced in the i 'th copy by a comma-separated sequence of $i - 1$ copies of the subexpression (indexed by j) with two transformations applied to each copy of the subexpression:
 - Any sub-subexpression that is a monogram and is not underlined is replaced by an underlined copy of that monogram with an additional subscript j appended, where j is the number of the copy of the subexpression (starting from the left with 1). (The added underline is removed again in step 3 below; it prevents step 2 below from operating on this monogram.)
 - Any subexpression that is underlined one or more times is replaced by a copy of that subexpression with one underline removed.
2. Next, any subexpression that is a monogram and is not underlined is replaced by a copy of that monogram with an additional subscript i appended, where i is the number of the copy (starting from the left with 1).
3. Finally, any subexpression that is underlined one or more times is replaced by a copy of that subexpression with one underline removed.

Thus $\overline{[\tau/P]\tau'}$ means $[\tau/P]\tau'_1, [\tau/P]\tau'_2, \dots, [\tau/P]\tau'_n$, so one possible concrete expansion of this expression to 5 copies is $[\tau/P]\tau'_1, [\tau/P]\tau'_2, [\tau/P]\tau'_3, [\tau/P]\tau'_4, [\tau/P]\tau'_5$; likewise, $x + \overline{h(\overline{x})}$ might expand to, say, four copies, producing $x_1 + h(), x_2 + h(x_1), x_3 + h(x_1, x_2), x_4 + h(x_1, x_2, x_3)$. The expression $\overline{(x, \underline{x})}$ can expand into all possible pairs of two distinct items out of n , preserving order, for example $(x_1, x_2), (x_1, x_3), (x_2, x_3), (x_1, x_4), (x_2, x_4), (x_3, x_4)$.

If an overbar or arrow construction expands to zero copies and there is a comma adjacent to the construction (or perhaps two, one on each side), then exactly one adjacent comma is deleted.

If overbar constructions are nested, they are expanded outermost first. Therefore the shorthand $f[\overline{P <: \{\bar{\tau}\}}]$ means $f[\overline{P_1 <: \{\bar{\tau}_1\}, P_2 <: \{\bar{\tau}_2\}, \dots, P_n <: \{\bar{\tau}_n\}}]$, which in turn means:

$$\begin{aligned} f[\overline{P_1 <: \{\tau_{11}, \tau_{12}, \dots, \tau_{1n_1}\}, \\ P_2 <: \{\tau_{21}, \tau_{22}, \dots, \tau_{2n_2}\}, \\ \dots, \\ P_n <: \{\tau_{n1}, \tau_{n2}, \dots, \tau_{nn_n}\}}] \end{aligned}$$

so one possible concrete expansion is:

$$\begin{aligned} f[\overline{P_1 <: \{\tau_{11}, \tau_{12}, \tau_{13}\}, \\ P_2 <: \{\tau_{21}\}, \\ P_3 <: \{\}, \\ P_4 <: \{\tau_{41}, \tau_{42}, \tau_{43}, \tau_{44}, \tau_{45}, \tau_{46}\}}] \end{aligned}$$

Note the use of whitespace between subscripts so that τ_{412} is clearly different from τ_{412} .

The function $\#$ returns an integer saying how many arguments it was given; thus $\#(\bar{x})$ tells the length of the sequence into which $\bar{x}$ has expanded.

After all occurrences of the overbar construction have been expanded, three other shorthand substitutions take place, in the following order:

- Every occurrence of a BNF nonterminal and every monogram whose base letter has been described as *ranging over* a BNF nonterminal of a specified grammar is replaced by a token sequence generated by the grammar from that nonterminal.
- Each occurrence of the symbol “ $_$ ” is replaced by any sequence of tokens that is correctly balanced with respect to enclosing pairs of all kinds, such that every comma and semicolon in the sequence is contained within at least one matched pair of enclosers. (This is used as a “don’t care” indication when asking whether any of a set of constructs matches a certain syntactic pattern. Note that an underline differs from a “don’t care” indication in two ways: an underline is below the baseline rather than on the baseline, and an underline always has nonblank symbols above it.) If a “ $_$ ” expands to an empty token sequence and there is a comma adjacent to the “ $_$ ” (or perhaps two, one on each side), then exactly one adjacent comma is deleted.
- Each occurrence of the symbol “ $\bullet$ ” is deleted. (This symbol is used as an explicit indication that an empty sequence of symbols is intended.)

When any of these shorthands is used in an overall context (such as a BNF rule, inference rule, axiom, or expository sentence or paragraph), it is as if there were an infinite number of instantiations of that context, one for each possible expansion of the shorthand. Two consistency constraints must

be obeyed in performing the substitutions for any single such context:

- If the same monogram (with identical decorations) has an additional subscript attached to it by more than one overbar construction, then all such overbar constructions are constrained to produce the same number of copies in any given instantiation of the rule; otherwise the choices for the number of copies produced by each overbar construction are free and independent.
- If the base letter of a monogram ranges over a BNF nonterminal, then multiple identical occurrences of the monogram must be replaced by identical copies of a single generated token sequence.

The last two constraints rely on metavariable declarations such as those in Figure 2. A declaration such as “ e ranges over expressions e ” means “monograms with base letter e expand into expressions generated by BNF nonterminal e ”; this may seem redundant, but only because by convention we frequently use a single-letter identifier as a BNF nonterminal and then go on to use that same single-letter identifier as a base letter for monograms. A declaration such as “ $\alpha, \gamma, \rho, \chi, \eta$ range over lattice types α ” means “monograms with base letter α or γ or ρ or χ or η expand into expressions generated by BNF nonterminal α ” which is more clearly not a redundant statement.

As an additional convenience using these shorthands:

- If a judgment has several comma-separated expressions to the right of the turnstile “ $\vdash$ ”, it is as if there were several distinct judgments, one containing each of the expressions to the right of the turnstile. Thus the judgment $\Gamma \vdash \tau_1 <: \tau'_1, \tau_2 <: \tau'_2, \tau_3 <: \tau'_3$ means the same as three separately written judgments:

$$\Gamma \vdash \tau_1 <: \tau'_1 \quad \Gamma \vdash \tau_2 <: \tau'_2 \quad \Gamma \vdash \tau_3 <: \tau'_3$$

- If a judgment has nothing to the right of the turnstile, it is as if there were no judgment written at all.
- If an inference rule has several comma-separated expressions or judgments as premises, it is as if the premises had been separated by the customary wide whitespace (and thus treated as conjuncts).
- If an inference rule has multiple consequents or several comma-separated expressions or judgments as consequents, it is as if there were several distinct inference rules, one containing each of the consequents.
- If an inference rule has no consequents, it is as if there were no inference rule written at all.
- If an inference rule has portions enclosed in framed boxes, then there is a choice when instantiating it: either include the contents of all framed boxes, or omit the content of all framed boxes.

As an extreme (but useful) example of the application of these conventions, consider this axiom:

$$\Delta \vdash \overline{E[\pi(\overline{v})]} \longrightarrow \overline{E[v]}$$

In order to apply this axiom to a particular case, we may freely choose to expand the largest overbar construction to produce, say, two copies:

$$\Delta \vdash E[\pi_1(\overline{v})] \longrightarrow E[v_1], E[\pi_2(\overline{v})] \longrightarrow E[v_2]$$

Note that both the π symbol and the second occurrence of v receive subscripts in each copy, but the underlines (which are removed as part of the expansion process) prevent the first occurrence of v (which happens to have a second overbar) and the two occurrences of E from receiving subscripts. Now we expand the remaining overbars, but because they will attach subscripts to the monogram v , and v has already had subscripts attached by the larger overbar, we must choose the same number of copies (two) for each of these overbars:

$$\Delta \vdash E[\pi_1(v_1, v_2)] \longrightarrow E[v_1], E[\pi_2(v_1, v_2)] \longrightarrow E[v_2]$$

Then this judgment with two comma-separated expressions to the right of the turnstile is understood to mean two distinct judgments:

$$\begin{aligned} \Delta \vdash E[\pi_1(v_1, v_2)] &\longrightarrow E[v_1] \\ \Delta \vdash E[\pi_2(v_1, v_2)] &\longrightarrow E[v_2] \end{aligned}$$

We use the conventional notation $\overline{[S/Id]}Y$ to indicate a copy of string Y (delimited on the right by commas, semicolons, and right enclosers) modified by simultaneous substitution of a copy of string S_1 for each occurrence of Id_1 , a copy of string S_2 for each occurrence of Id_2 , and so on; consecutive substitution constructs are processed from right to left. The substitution is permitted only if names of variables bound in $\overline{[S/Id]}$, but alpha-conversion (systematic renaming of such bound variables to fresh names) may be performed on Y in order to enable the substitution to take place. We use the conventional notation $E[e]$ to mean an evaluation context with its “hole” replaced by the expression e (that is, $[e/\square]E$).

A final note: we sometimes use enclosers of different sizes within an expression purely to enhance readability; the size of such a symbol does not affect its meaning in the formalism.

B. Proofs about simplifying a constructed type of a union or intersection type

B.1 Covariant parameter receives intersection type

Consider a trait $G[\text{covariant } P]$; we wish to show that $G[\alpha \cap \eta] \equiv G[\alpha] \cap G[\eta]$. We do this by showing that $G[\alpha \cap \eta] <: G[\alpha] \cap G[\eta]$, and $G[\alpha] \cap G[\eta] <: G[\alpha \cap \eta]$.

(i) G is covariant and $\alpha \cap \eta <: \alpha$, so $G[\alpha \cap \eta] <: G[\alpha]$. Similarly, $\alpha \cap \eta <: \eta$, so $G[\alpha \cap \eta] <: G[\eta]$. From the definition of $\cap$, $G[\alpha \cap \eta] <: G[\alpha] \cap G[\eta]$.

(ii) Suppose $G[\alpha] \cap G[\eta] \not<: G[\alpha \cap \eta]$. Then there must be some value v that belongs to type $G[\alpha] \cap G[\eta]$ but not to type $G[\alpha \cap \eta]$; therefore $\text{ilk}(v) <: G[\alpha] \cap G[\eta]$ and $\text{ilk}(v) \not<: G[\alpha \cap \eta]$, and $\text{ilk}(v) <: G[\alpha] \cap G[\eta]$ implies both $\text{ilk}(v) <: G[\alpha]$ and $\text{ilk}(v) <: G[\eta]$. By the Ancestor Meet Rule, there must exist some χ such that $\text{ilk}(v) <: G[\chi]$ and $G[\chi] <: G[\alpha]$ and $G[\chi] <: G[\eta]$. Because G is covariant, $\chi <: \alpha$ and $\chi <: \eta$; therefore $\chi <: \alpha \cap \eta$; therefore $G[\chi] <: G[\alpha \cap \eta]$. But $\text{ilk}(v) <: G[\chi]$, so by transitivity $\text{ilk}(v) <: G[\alpha \cap \eta]$. That means v belongs to type $G[\alpha \cap \eta]$, a contradiction. Our original assumption was incorrect, therefore $G[\alpha] \cap G[\eta] <: G[\alpha \cap \eta]$.

B.2 Contravariant parameter receives union type

Consider a trait $G[\text{contravariant } P]$; we wish to show $G[\alpha \cup \eta] \equiv G[\alpha] \cap G[\eta]$. We do this by showing that $G[\alpha \cup \eta] <: G[\alpha] \cap G[\eta]$, and $G[\alpha] \cap G[\eta] <: G[\alpha \cup \eta]$.

(i) G is contravariant and $\alpha <: \alpha \cup \eta$, so $G[\alpha \cup \eta] <: G[\alpha]$. Similarly, $\eta <: \alpha \cup \eta$, so $G[\alpha \cup \eta] <: G[\eta]$. From the definition of $\cap$, $G[\alpha \cup \eta] <: G[\alpha] \cap G[\eta]$.

(ii) Suppose $G[\alpha] \cap G[\eta] \not<: G[\alpha \cup \eta]$. Then there must be some value v that belongs to type $G[\alpha] \cap G[\eta]$ but not to type $G[\alpha \cup \eta]$; therefore $\text{ilk}(v) <: G[\alpha] \cap G[\eta]$ and $\text{ilk}(v) \not<: G[\alpha \cup \eta]$, and $\text{ilk}(v) <: G[\alpha] \cap G[\eta]$ implies both $\text{ilk}(v) <: G[\alpha]$ and $\text{ilk}(v) <: G[\eta]$. By the Ancestor Meet Rule, there must exist some χ such that $\text{ilk}(v) <: G[\chi]$ and $G[\chi] <: G[\alpha]$ and $G[\chi] <: G[\eta]$. Because G is contravariant, $\alpha <: \chi$ and $\eta <: \chi$; therefore $\alpha \cup \eta <: \chi$; therefore $G[\chi] <: G[\alpha \cup \eta]$. But $\text{ilk}(v) <: G[\chi]$, so by transitivity $\text{ilk}(v) <: G[\alpha \cup \eta]$. That means v belongs to type $G[\alpha \cup \eta]$, a contradiction. Our original assumption was incorrect, therefore $G[\alpha] \cap G[\eta] <: G[\alpha \cup \eta]$.